INDEX

Name : Ishwarya. L Std. : AI & DSec. :

Roll No. : Sub :

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	23/1/26	Network commands used in Linux		
2.	25/1/26	Network topology using hub, switch and router		
3.	1/8/26	Study of different types of network cables		
4.	1/8/26	Configuration of LAN		
5.	4/8/26	Packet capture using wireshark		
6.	8/8/26	Error detection & correction using Hamming code		
7.	8/8/26	Flow chart control at data link layer using sliding window		
8a).		Discover live hosts using Nmap		
8b).		configuration of wireless LAN using cisco packet tracer		
9.		Implementation of subnetting in cisco Packet Tracer		
10a)		Internetworking with Router		
10b).		Wireless router, DHCP server and internet cloud		

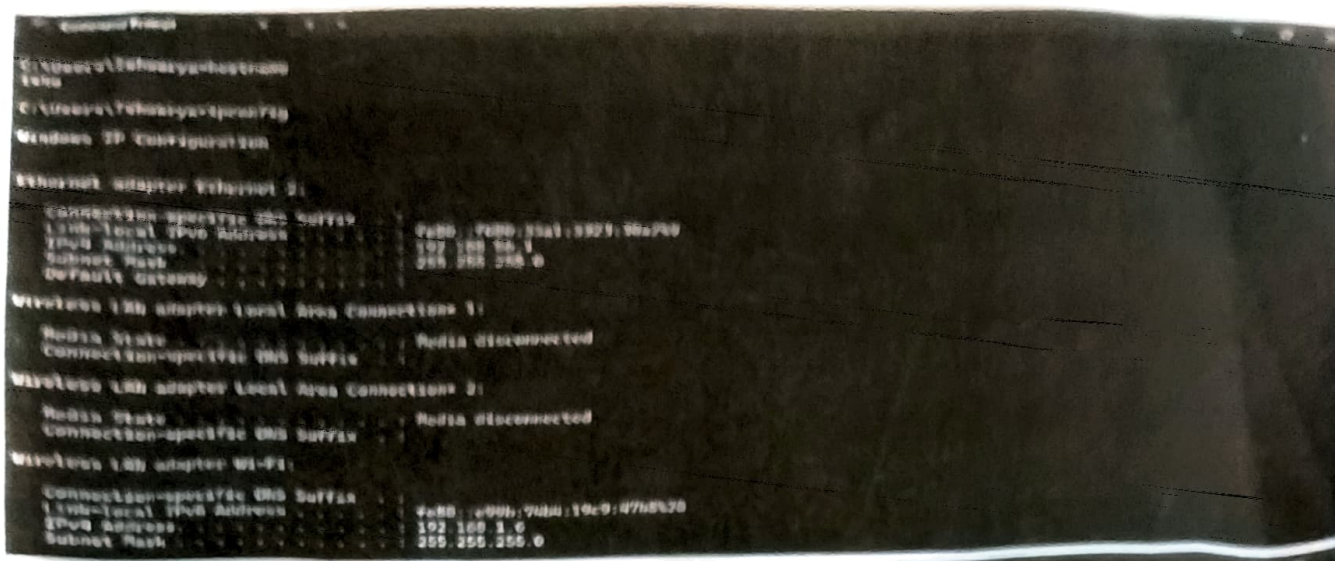
INDEX

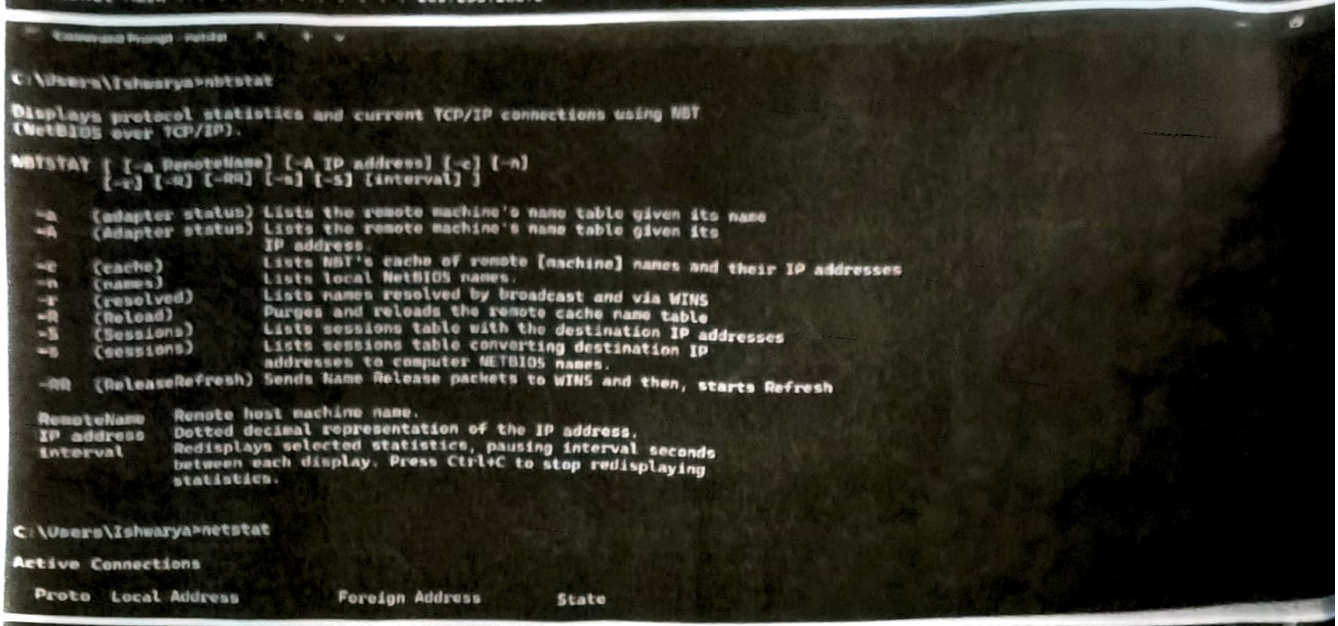
Name Ishwarya. L Class AI&DS Div. Roll No

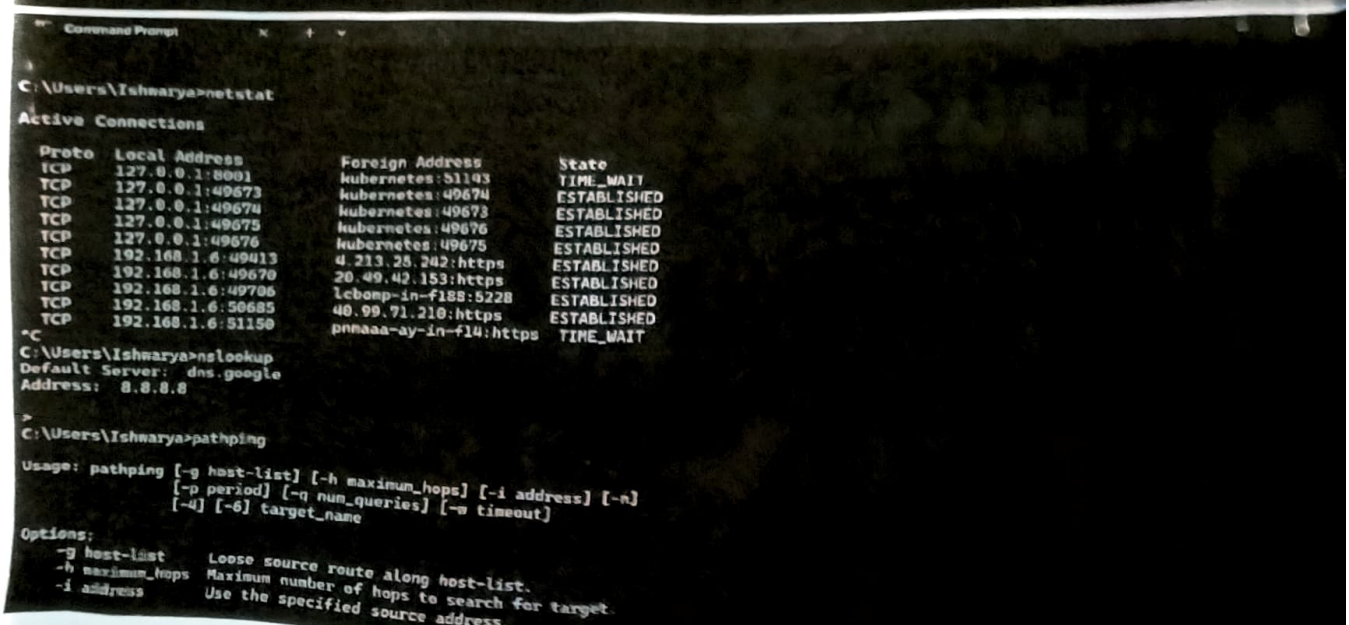
School Name Sub

S. No	Date	Particular	Page No	Signature
11 a).		Static Routing configuration - cisco Packet Tracer		
11 b).		Routing Information Protocol		
12 a)		Echo client-server		
12 b)		chat client-server		
13.		Implement own ping program		
14.		Packet Sniffing using raw sockets		
15.		Analysis of various types of servers using weblalizer		

completed


Command Prompt
C:\Users\Ishwarya>hostname
DESKTOP-
C:\Users\Ishwarya>ipconfig
Windows IP Configuration
Ethernet adapter Ethernet 2:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Wireless LAN adapter Local Area Connection* 1:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Wireless LAN adapter Local Area Connection* 2:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Wireless LAN adapter Wi-Fi:
Connection-specific DNS Suffix . :
Link-local IPv6 Address . . . . . : fe80::c00b:7d8d:19c9:47b8%20
IPv4 Address . . . . . : 192.168.1.6
Subnet Mask . . . . . : 255.255.255.0


C:\Users\Ishwarya>nbtstat
Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).
NBTSTAT [[-a RemoteName] [-A IP address] [-c] [-n]
[-r] [-R] [-RR] [-s] [-S] [interval]]
-a (adapter status) Lists the remote machine's name table given its name
-A (Adapter status) Lists the remote machine's name table given its
IP address.
-c (cache) Lists NBT's cache of remote [machine] names and their IP addresses
-n (names) Lists local NetBIOS names.
-r (resolved) Lists names resolved by broadcast and via WINS
-R (Reload) Purges and reloads the remote cache name table
-S (Sessions) Lists sessions table with the destination IP addresses
-s (sessions) Lists sessions table converting destination IP
addresses to computer NETBIOS names.
-RR (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh
RemoteName Remote host machine name.
IP address Dotted decimal representation of the IP address.
interval Redisplays selected statistics, pausing interval seconds
between each display. Press Ctrl+C to stop redisplaying
statistics.
C:\Users\Ishwarya>netstat
Active Connections
Proto Local Address Foreign Address State


Command Prompt
C:\Users\Ishwarya>netstat
Active Connections
Proto Local Address Foreign Address State
TCP 127.0.0.1:8001 kubernetes:51143 TIME_WAIT
TCP 127.0.0.1:49673 kubernetes:49674 ESTABLISHED
TCP 127.0.0.1:49674 kubernetes:49673 ESTABLISHED
TCP 127.0.0.1:49675 kubernetes:49676 ESTABLISHED
TCP 127.0.0.1:49676 kubernetes:49675 ESTABLISHED
TCP 192.168.1.6:49413 4.213.25.242:https ESTABLISHED
TCP 192.168.1.6:49670 20.49.42.153:https ESTABLISHED
TCP 192.168.1.6:49708 lcbomp-in-f188:5228 ESTABLISHED
TCP 192.168.1.6:50685 40.99.71.210:https ESTABLISHED
TCP 192.168.1.6:51150 pnmaaa-ay-in-f14:https TIME_WAIT
^C
C:\Users\Ishwarya>nslookup
Default Server: dns.google
Address: 8.8.8.8
C:\Users\Ishwarya>pathping
Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
[-p period] [-q num_queries] [-w timeout]
[-4] [-6] target_name
Options:
-g host-list Loose source route along host-list.
-h maximum_hops Maximum number of hops to search for target.
-i address Use the specified source address.

Command Prompt

```
  interface     the interface number for the specified route.
  METRIC        specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*            .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
              destination^      ^mask      ^gateway     metric^    ^
                                                        Interface^
      If IF is not given, it tries to find the best interface for a given
      gateway.
    > route ADD 3ffe::/32 3ffe::1
```

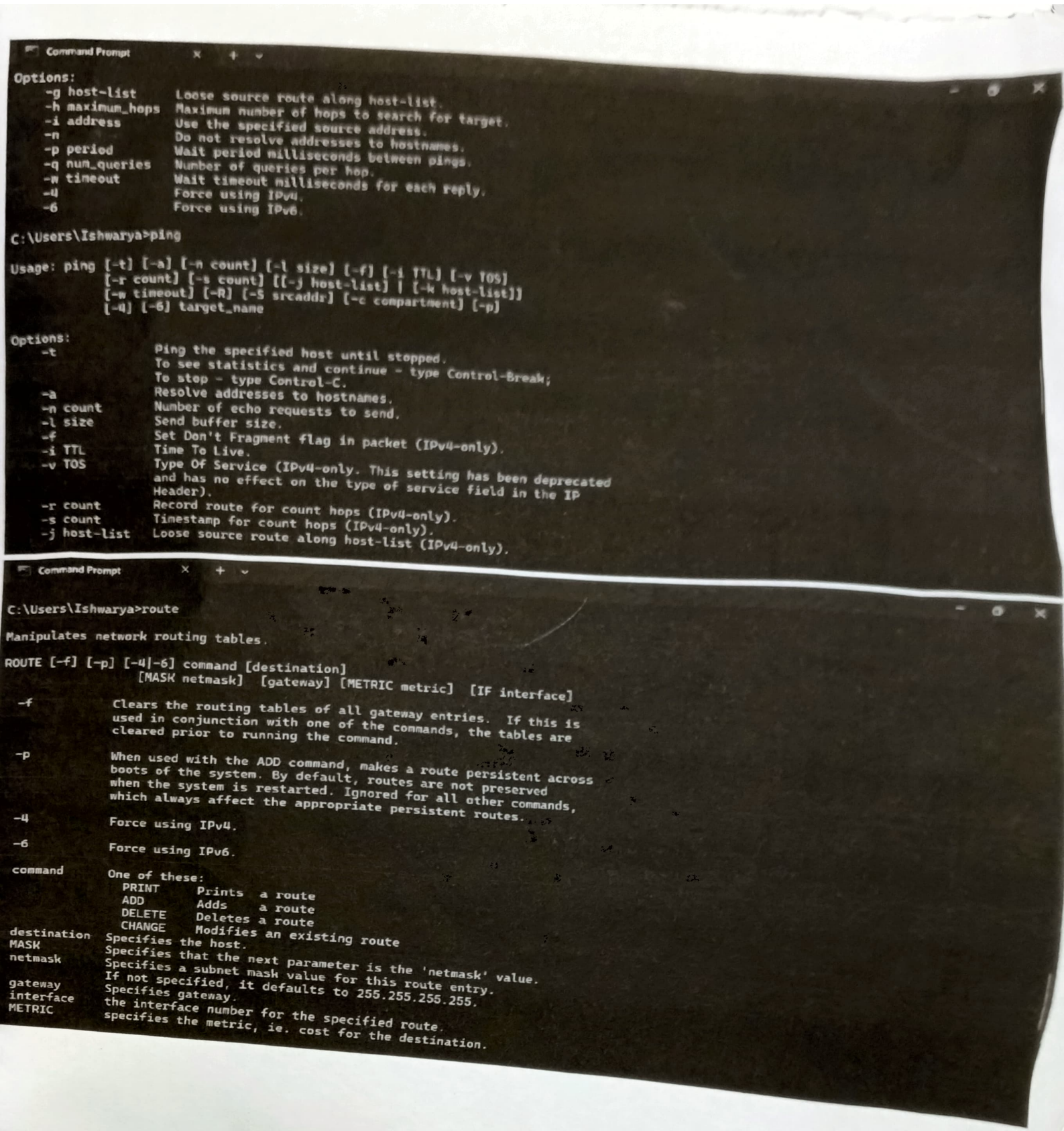
Command Prompt

```
C:\Users\Ishwarya>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens. The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.
Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
```



Command Prompt

```
Options:
    -g host-list      Loose source route along host-list.
    -h maximum_hops   Maximum number of hops to search for target.
    -i address        Use the specified source address.
    -n                Do not resolve addresses to hostnames.
    -p period         Wait period milliseconds between pings.
    -q num_queries    Number of queries per hop.
    -w timeout        Wait timeout milliseconds for each reply.
    -4                Force using IPv4.
    -6                Force using IPv6.

C:\Users\Ishwarya>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t                Ping the specified host until stopped.
                      To see statistics and continue - type Control-Break;
                      To stop - type Control-C.
    -a                Resolve addresses to hostnames.
    -n count          Number of echo requests to send.
    -l size           Send buffer size.
    -f                Set Don't Fragment flag in packet (IPv4-only).
    -i TTL            Time To Live.
    -v TOS            Type Of Service (IPv4-only. This setting has been deprecated
                      and has no effect on the type of service field in the IP
                      Header).
    -r count          Record route for count hops (IPv4-only).
    -s count          Timestamp for count hops (IPv4-only).
    -j host-list      Loose source route along host-list (IPv4-only).
```

Command Prompt

```
C:\Users\Ishwarya>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f              Clears the routing tables of all gateway entries.  If this is
                  used in conjunction with one of the commands, the tables are
                  cleared prior to running the command.

  -p              When used with the ADD command, makes a route persistent across
                  boots of the system. By default, routes are not preserved
                  when the system is restarted. Ignored for all other commands,
                  which always affect the appropriate persistent routes.

  -4              Force using IPv4.

  -6              Force using IPv6.

  command         One of these:
                    PRINT     Prints  a route
                    ADD       Adds    a route
                    DELETE    Deletes a route
                    CHANGE    Modifies an existing route
  destination     Specifies the host.
  MASK            Specifies that the next parameter is the 'netmask' value.
  netmask         Specifies a subnet mask value for this route entry.
                  If not specified, it defaults to 255.255.255.255.
  gateway         Specifies gateway.
  interface       the interface number for the specified route.
  METRIC          specifies the metric, ie. cost for the destination.
```

23/7/2026 1. NETWORK COMMANDS USED IN LINUX

Aim:

To study various network commands used in linux and windows.

Commands:

(i) arp - Address Resolution Protocol
↳ used to display system's IP address and sub-commands.

(ii) hostname - displays name of the current system connected to network.

(iii) ipconfig - displays computer's IP address and other network configuration details.

(iv) nbstat - Shows NetBIOS over TCP/IP information

(v) netstat - shows active network connection/statistics

(vi) nslookup - Finds the IP address of a domain name using DNS server

(vii) pathping - checks network path & identifies packet loss.

(viii) ping - Tests whether a remote host (or) device reachable

(ix) route - displays and manages the system's routing table

Result:

Thus, the network commands has been studied and implemented successfully in a terminal.

25/7/2026

2. NETWORK TOPOLOGY USING HUB, SWITCH AND ROUTER

Aim:

To create and simulate simple network topologies using a hub, switch & router in cisco packet tracer and study data connection between devices.

Procedure:

a). Hub Topology:

(i) opened cisco packet tracer.

(ii) Placed one hub and three PC's in the workplace.

(iii) Assign IP address to all the three PC's

(iv) create a connection between hub and all the personal computer through copper wire.

(v) Select simple PDU (message icon) and click on the source PC and then double click on the destination PC.

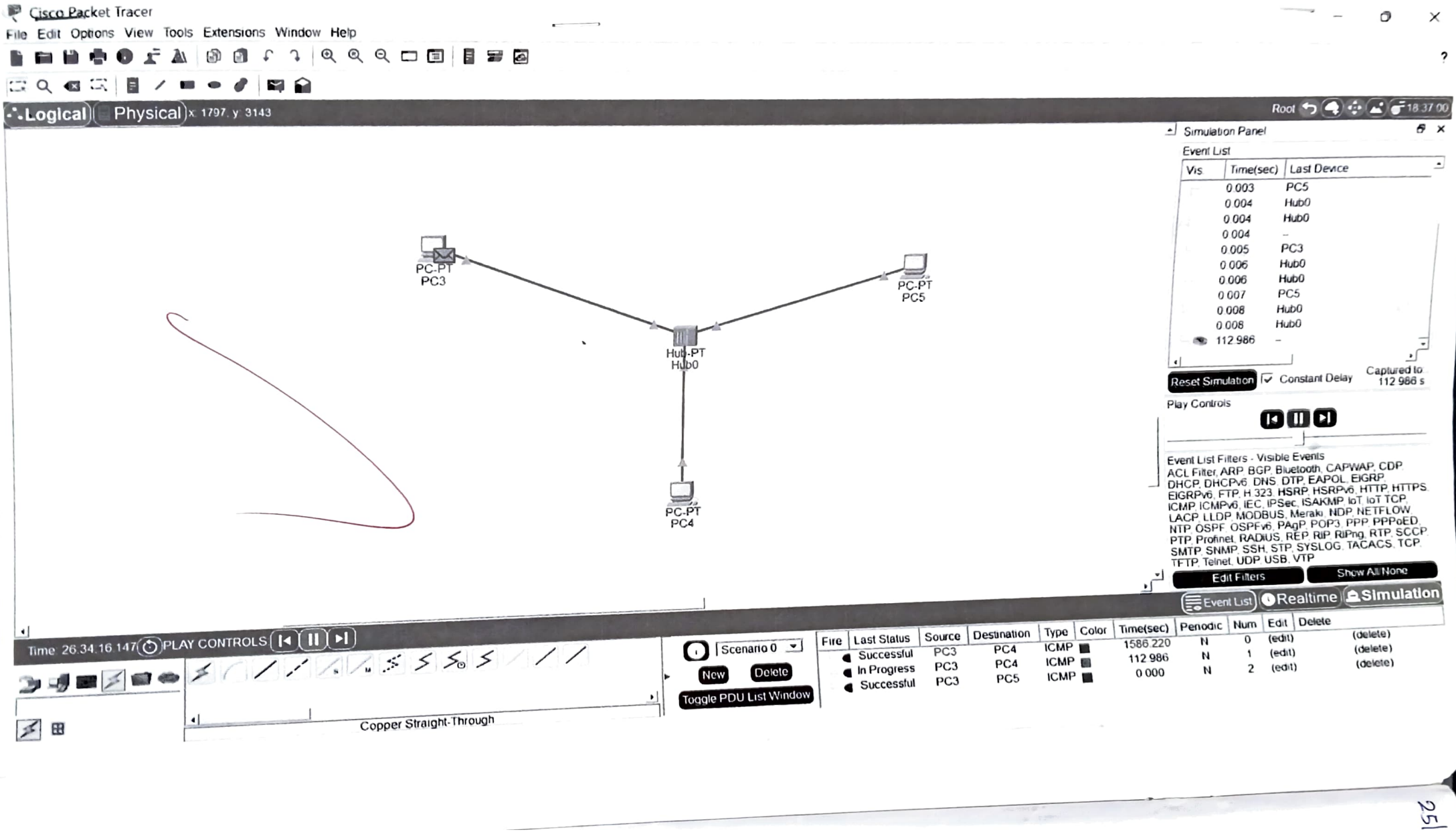
(vi) Select the simulation mode and observe the packet transmission and communication between PC's.

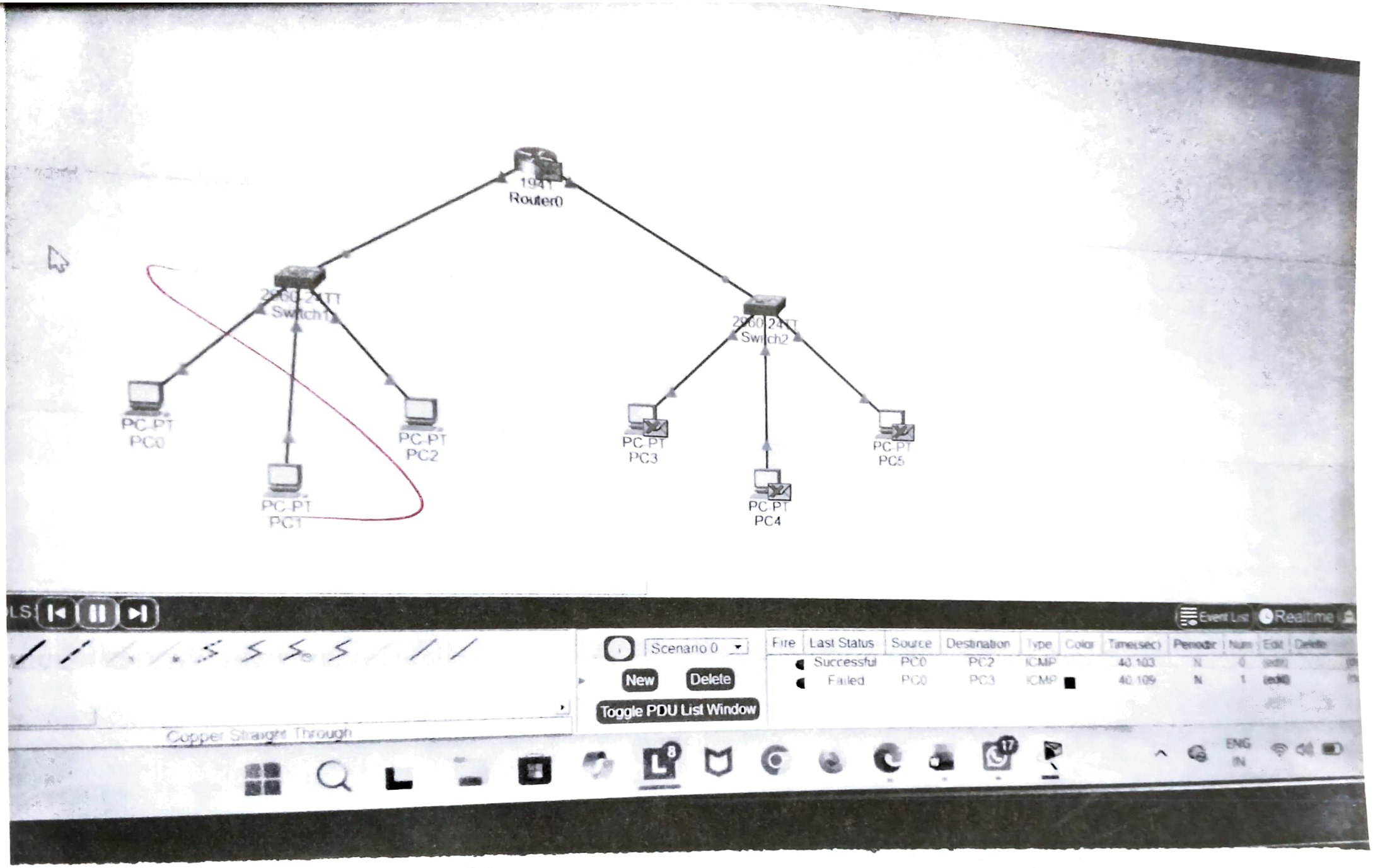
b). Switch and Router topology:

(i) open cisco packet tracer

(ii) Place ~~two~~ switches, one router and six PC's on the workplace.

(iii) connect each switch with the three PC's and then connect the two switches with the router.


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical Physical x: 1797, y: 3143
Root
PC-PT PC3
PC-PT PC5
Hub-PT Hub0
PC-PT PC4
Simulation Panel
Event List
Vis. Time(sec) Last Device
0.003 PC5
0.004 Hub0
0.004 Hub0
0.004 –
0.005 PC3
0.006 Hub0
0.006 Hub0
0.007 PC5
0.008 Hub0
0.008 Hub0
112.986 –
Reset Simulation
Constant Delay
Captured to: 112.986 s
Play Controls
Event List Filters - Visible Events
ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EAPOL, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IEC, IPSec, ISAKMP, IoT, IoT TCP, LACP, LLDP, MODBUS, Meraki, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, PPP, PPPoED, PTP, Profinet, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, VTP
Edit Filters
Show All/None
Event List
Realtime
Simulation
Time: 26.34.16.147
PLAY CONTROLS
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC3 PC4 ICMP 1586.220 N 0 (edit) (delete)
In Progress PC3 PC4 ICMP 112.986 N 1 (edit) (delete)
Successful PC3 PC5 ICMP 0.000 N 2 (edit) (delete)
Copper Straight-Through


1941
Router0
2960-24TT
Switch1
2960-24TT
Switch2
PC-PT
PC0
PC-PT
PC1
PC-PT
PC2
PC-PT
PC3
PC-PT
PC4
PC-PT
PC5
Event List
Realtime
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Successful
PC0
PC2
ICMP
40.103
N
0
Failed
PC0
PC3
ICMP
40.109
N
1
Copper Straight-Through
ENG
IN

(iv) Simulate the PDU (message icon) and click on the source PC and then double click on the destination PC.

(v) Select the simulation mode and observe the packet transmission and communication between pc's.

Result:

The network topologies using pc's, switch, route and hub have been implemented and the message has sent from source to destination through PDU.

18/2026 3 STUDY OF DIFFERENT TYPES OF NETWORK CABLES

a). understand different types of network cable:

Aim:

To study different types of network cable using UTP, STP, coaxial and fibre optic cable.

UTP: (Unshielded Twisted Pair cable)

Category :- category 3, category 5, category 5e

Maximum data transmit :- 10 bps, upto 100 Mbps, 1 Gbps

Advantages - ⇒ cheaper in cost, easy to install

disadvantages - More prone to EMI

uses - 10Base-T ethernet, Fast ethernet, Gigabit ethernet

STP: (Shielded Twisted Pair cable)

category - category 6, 6a

Maximum data transmit - 10 Gbps

SSTP:

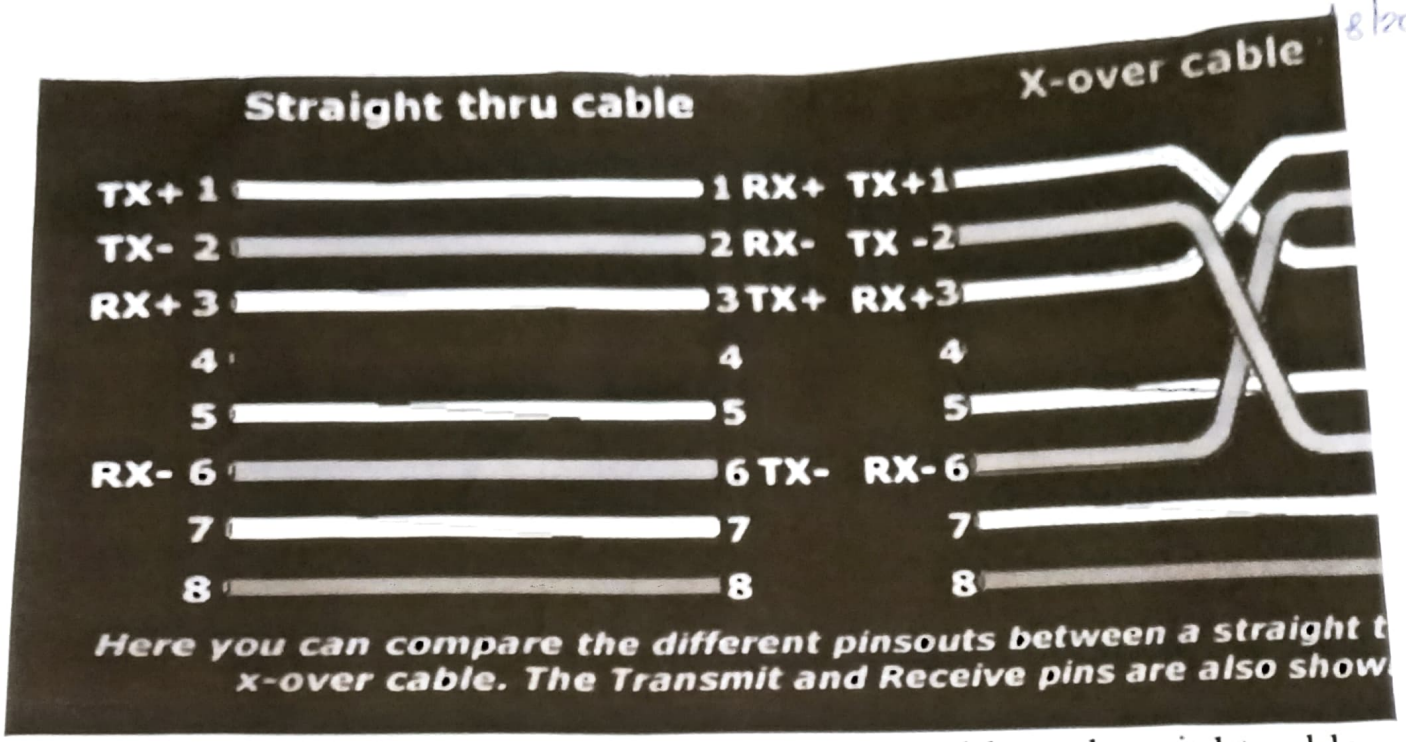
category - category 7

Maximum data transmit - 10 Gbps

Advantages - Shielded, faster than UTP, less susceptible to noise and interference

Disadvantages - Expensive, greater installation effort

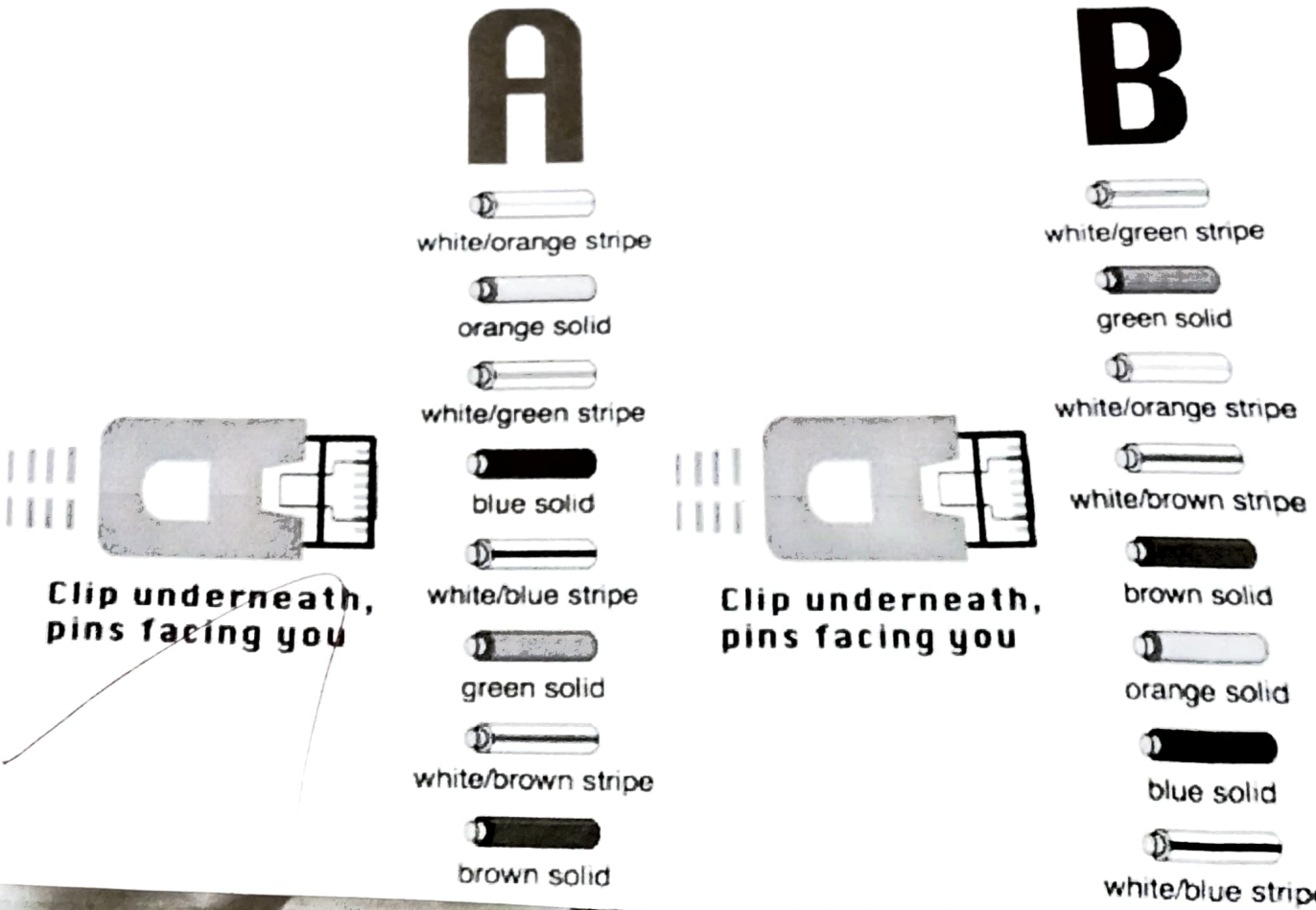
uses - Gigabit ethernet, 10 G ethernet (55 m), widely used in data centers

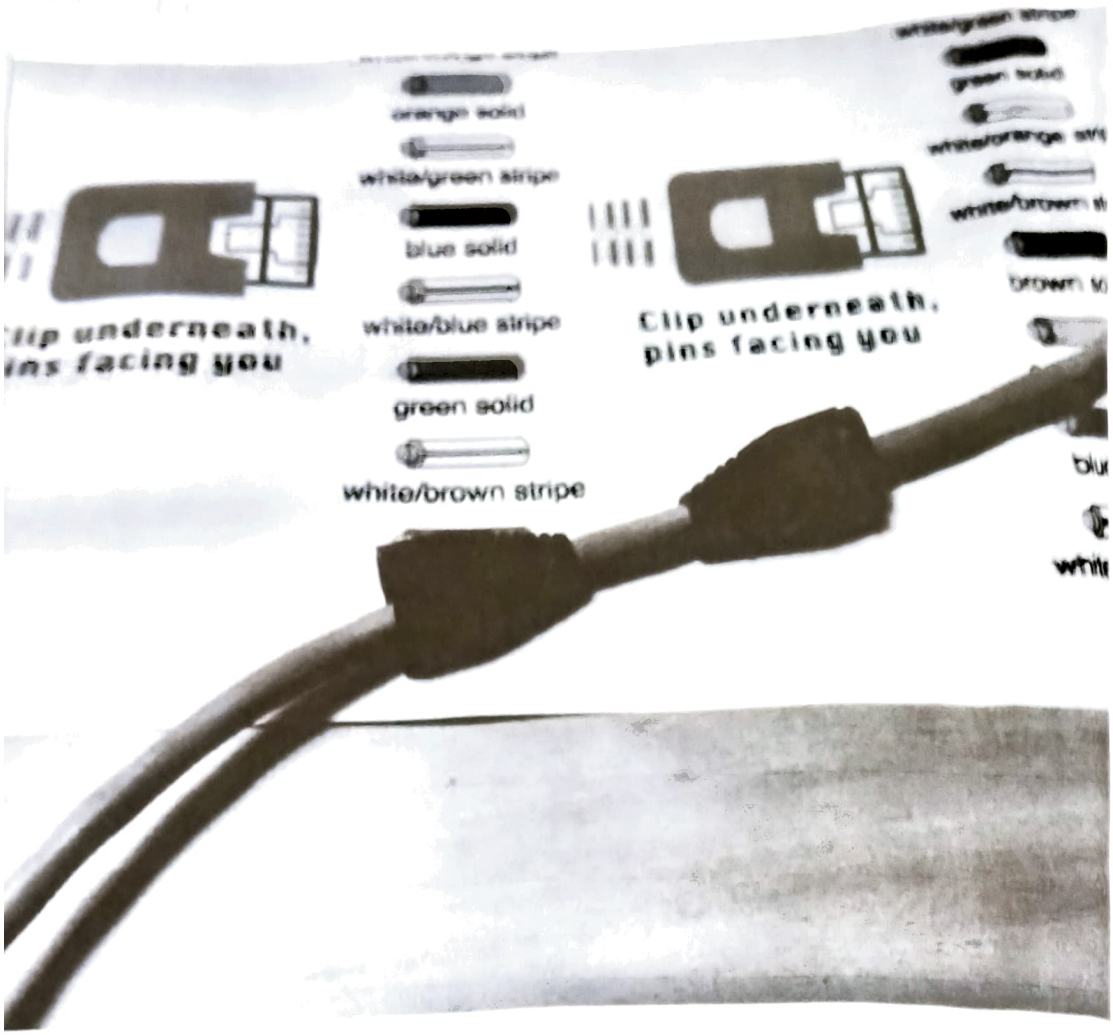


Difference between crossover cable and straight cable

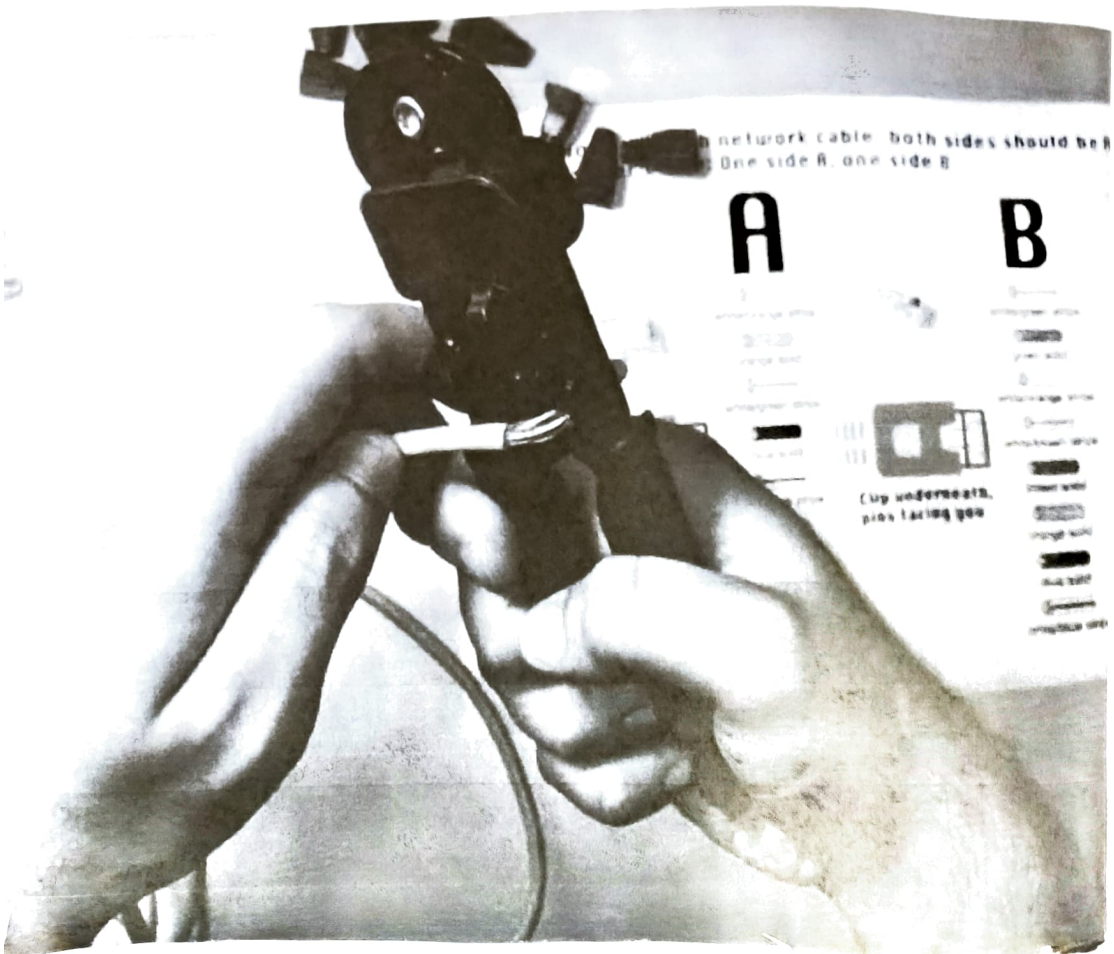
nt out the diagram below or have it handy as a reference

Straight through network cable: both sides should b
Crossover cable: One side A, one side B



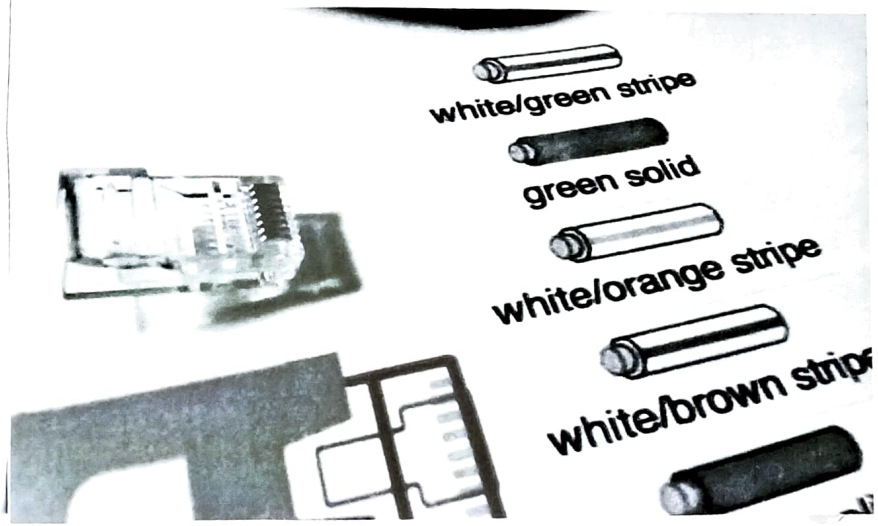


ext, strip approximately 1.5 cm of cable shielding from both ends
tool has a round area to complete this task.





4: Once the order is correct, bunch them together in a line, and if there are any out farther than others, snip them back to create an even level. The difficult a: cing these into the RJ45 plug without messing up the order. To do so, hold the the clip side facing away from you and have the gold pins facing toward you, a n.





5: After the wires are securely sitting inside the plug, insert it into the crimping ush down.

ıld be shaped correctly, but pushing too hard can crack the fragile plastic plug

coaxial cable:

category - RG-6, RG-59, RG-11

Maximum data transmit } - 10 - 100 Mbps

Advantages - High speed bandwidth, immune to interference, versatile

Disadvantages - Limited distance, size is bulky, cost

uses - Speed of signal is 500 m, television network, high speed internet connections.

Fibre optics cable:

category - Single mode, multi mode

Maximum data transmit } - 100 Gbps

Advantages - high speed, bandwidth, security

Disadvantages - Expensive, required skilled installers

uses - Maximum distance is 100 m

b). Make your own ethernet cross-over cable:

Tools needed:

(i) Ethernet cabling

(ii) crimping tool

(iii) Two RJ45 plugs

(iv) optional two plug shields.

Step 1: To start construction of device, begin by threading shields onto the cable.

Step 2: Next, strip approximately 1.5 cm of cable shielding from both ends. crimping tool has round area to complete this task

Step 3: After, you will need to untangle the wires; there should be four "twisted pairs". Referencing back to the sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B.

Step 4: once the order is correct, bunch them together in a line, and if there are any that stick out further from others, snip them back to create an even level.

Step 5: Next, push the cable right in. The notch at the end of the plug needs to be just over the cable shielding, and if it isn't, that means that you stripped off too much shielding.

Step 6: After the wires are securely sitting inside the plug, insert it into crimping tool and push down.

Step 7: Lastly, repeat for the other end using diagram B.

Result:

Thus, the different types of network cable has been studied along with their properties and uses.

1/8/2026 4. CONFIGURATION OF LAN

Aim:
To setup and configure a LAN (Local Area Network) using a switch and ethernet cables in your lab.

LAN: A Local Area Network (LAN) is a network that connects computer and other devices within a limited geographical area such as home, school (or) laboratory. It allows devices to communicate with each other & share resources

Procedure:
(i) connect 4 computers to a switch using ethernet cables.
(ii) Assigned IP addresses:
⇒ PC1 - 10.1.1.1
⇒ PC2 - 10.1.1.2
⇒ PC3 - 10.1.1.3
⇒ PC4 - 10.1.1.4
⇒ Subnet mask - 255.0.0.0
(iii) configure the switch with IP address 10.1.1.5
(iv) verify connectivity using the Ping command.
(v) Share a folder and accused it from the other PC's on the LAN.

IP address table:

Device	IP address	Subnet mask
PC1	10.1.1.1	255.0.0.0
PC2	10.1.1.2	255.0.0.0
PC3	10.1.1.3	255.0.0.0
PC4	10.1.1.4	255.0.0.0
Switch	10.1.1.5	255.0.0.0

Result:

A Local Area Network (LAN) was successfully set up and configured using a switch and ethernet cable. connectivity between all devices was verified using the ping command and file sharing is tested successfully.

6/8/2026

5. PACKET CAPTURE USING WIRESHARK

Aim:

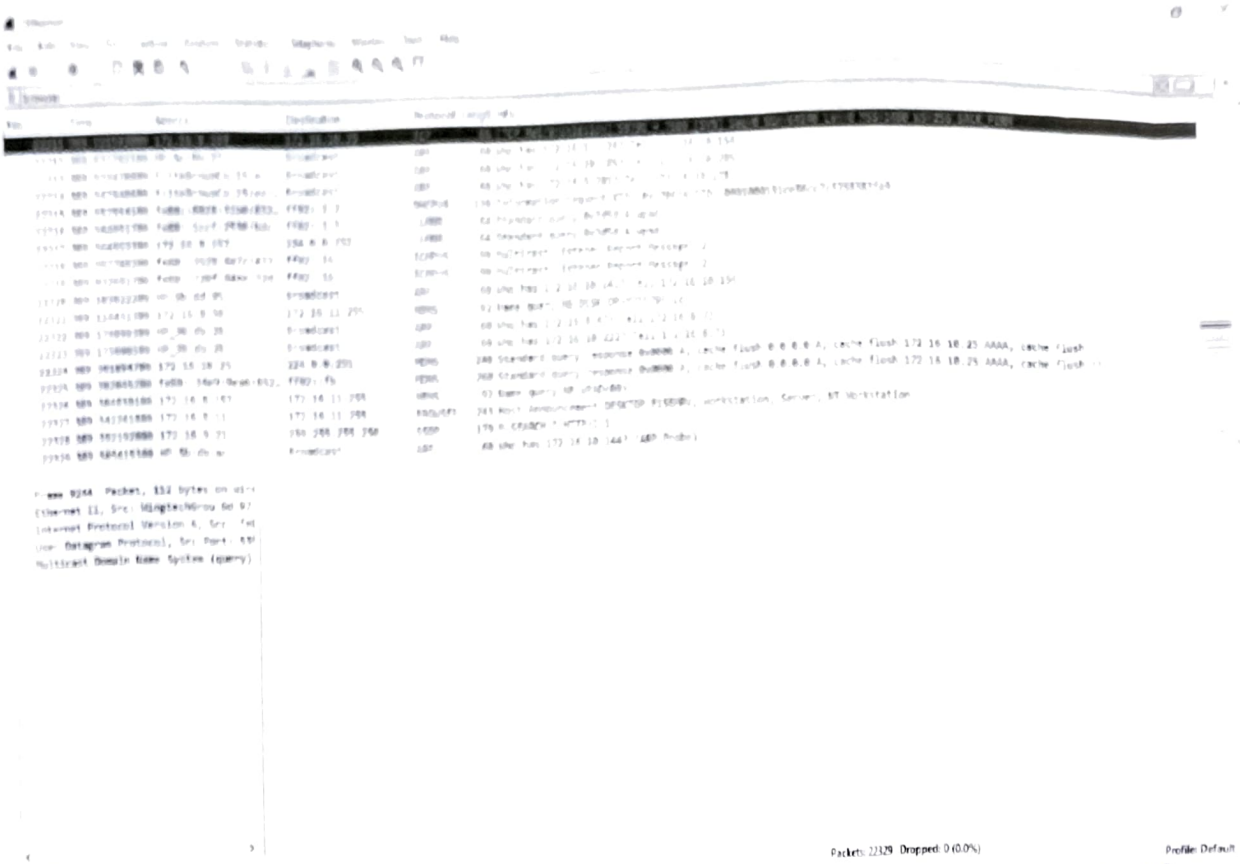
To capture filter inspect and analyze network packets using the wireshark packet analyzer

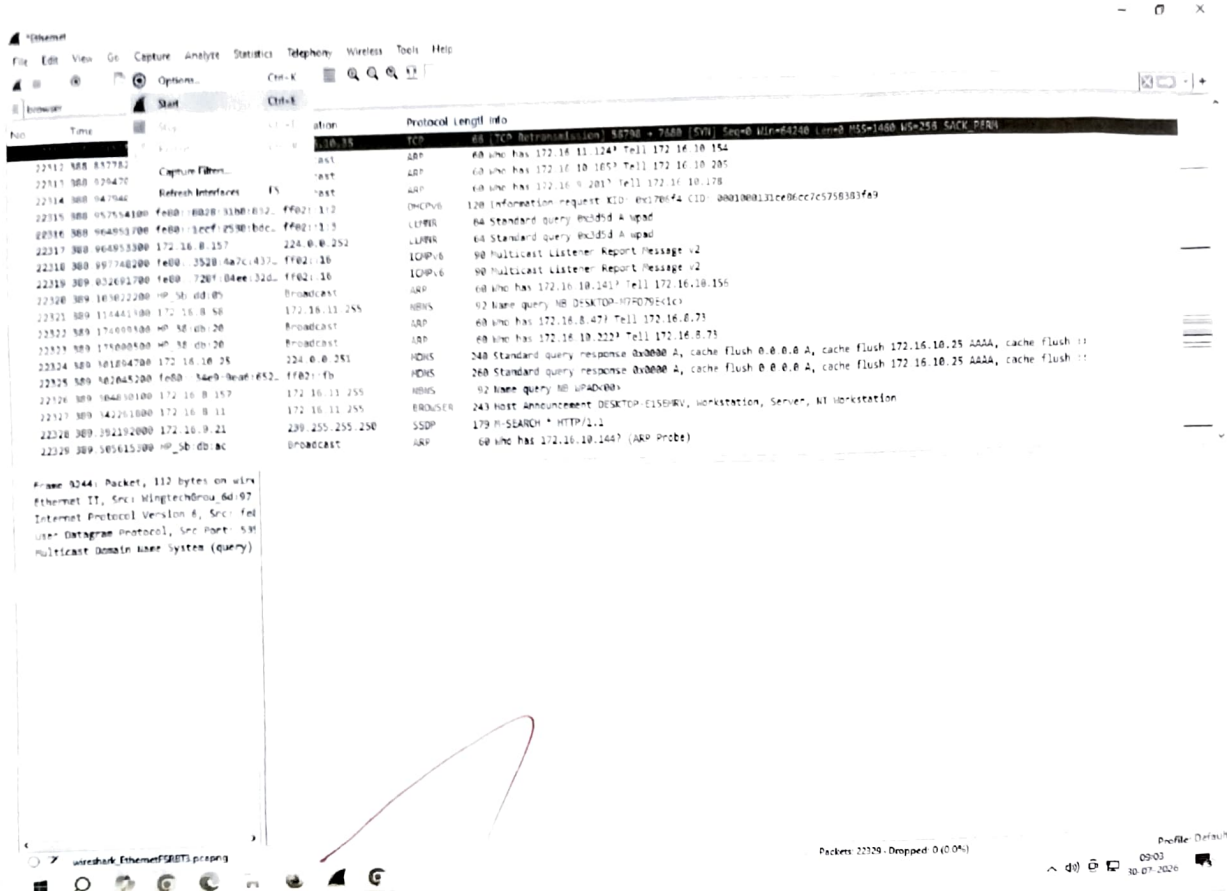
Algorithm:

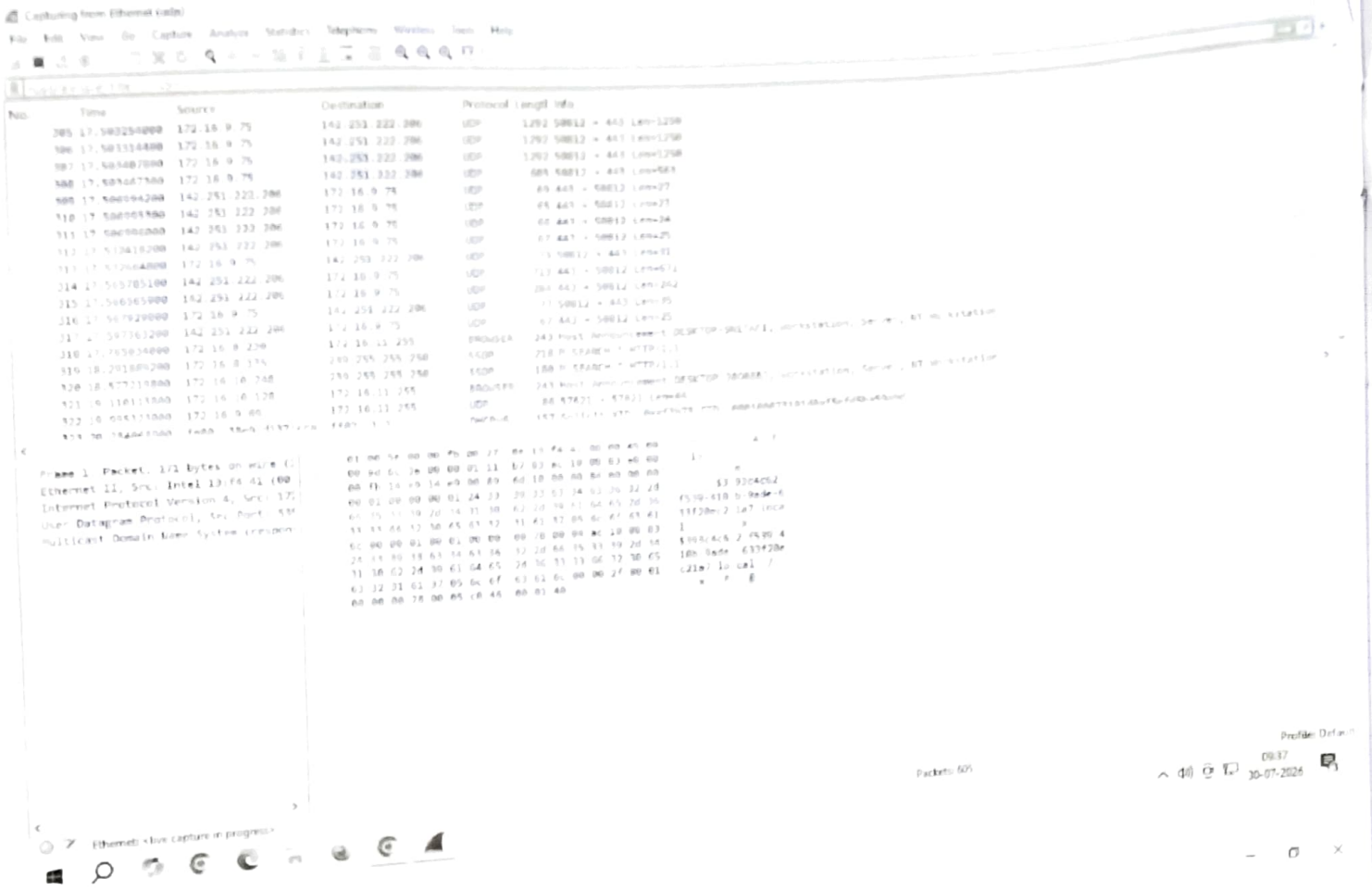
1). open wireshark
2). Select the active network interface
3). configure capture to stop after 100 packets
4). Start packet capture
5). Apply display filters
6). Inspect packet details and view the flow graph
7). Save the captured packets.

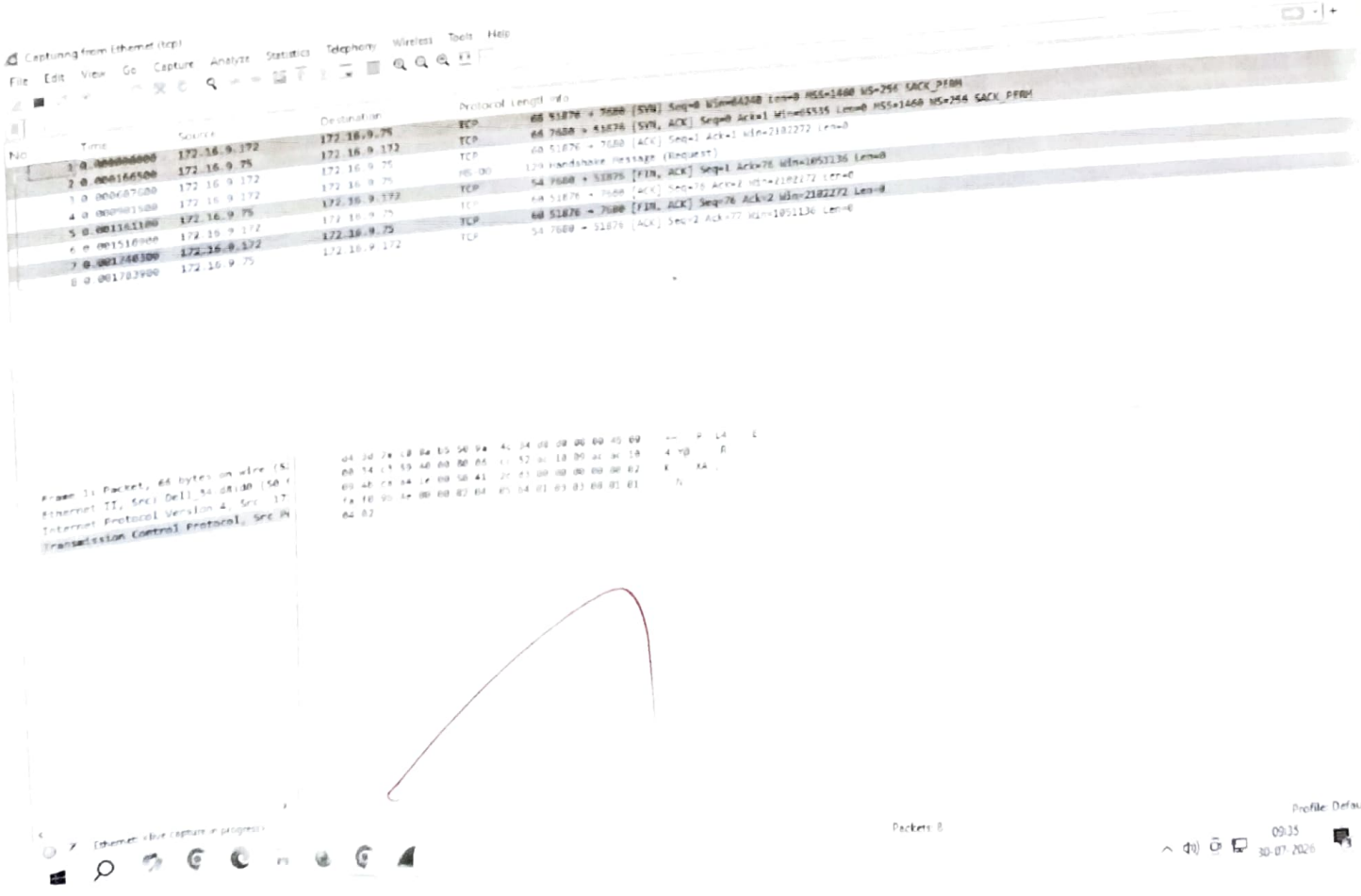
Procedure:

⇒ Launch wireshark and subset the active network interface.
⇒ Go to capture → options and set stop capture of 100 packets.
⇒ click start to design packet capture.
⇒
- TCP (UDP) : tcp || udp
- ARP : arp
- DNS : dns
- HTTP : http
- DHCP : bootp









⇒ Inspect the filtered packets in the packet list, details and bytes panes

⇒ Save the capture file for future analysis

Result:

Successfully captured 100 network packets, applied display filters for TCP, UDP, ARP, DNS, HTTP, IP, ICMP, DTCP analyzed packet details using wireshark.

python code:

```
# Hamming code
def hamming_encode(data):
    h = [0] * 13

    h[3], h[5], h[6], h[7] = map(int, data[:4])
    h[9], h[10], h[11], h[12] = map(int, data[4:])

    h[1] = h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11]
    h[2] = h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11]
    h[4] = h[5] ^ h[6] ^ h[7] ^ h[12]
    h[8] = h[9] ^ h[10] ^ h[11] ^ h[12]

    return ''.join(map(str, h[1:]))

def hamming_decode(code):
    h = [0] + list(map(int, code))

    s1 = h[1] ^ h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11]
    s2 = h[2] ^ h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11]
    s4 = h[4] ^ h[5] ^ h[6] ^ h[7] ^ h[12]
    s8 = h[8] ^ h[9] ^ h[10] ^ h[11] ^ h[12]

    error = s1 + 2*s2 + 4*s4 + 8*s8

    if error != 0:
        print("Error detected at position:", error)

        h[error] ^= 1
        print("Error corrected")
```

8/8/2026

6. ERROR DETECTION AND CORRECTION USING HAMMING CODE

Aim:

TO implement error detection and correction using Hamming code in Python and verify the correction of a single-bit error during data transmission.

Algorithm:

Sender:

(i) Read a text message from the user.

(ii) convert each character into its 8-bit binary representation.

(iii) Divide binary data into 8-bit blocks.

(iv) Add 4 redundant parity bits at positions 1, 2, 4 and 8

(v) Generate a 12-bit hamming code for each 8-bit block and store it as channel.txt

Receiver:

(i) Read the encoded data from channel.txt and divide it into 12-bit blocks.

(ii) calculate parity bits & find the error position.

(iii) If the error-position is non-zero, flip that bit to correct the error.

(iv) Remove the redundant bits and convert the remaining 8 bits into ASCII characters.

Hamming Encoding:

For 8 data bits, 4 parity bits required

total bits = 8 data bits + 4 parity bits
= 12 bits

position:

1	2	3	4	5	6	7	8	9	10	11	12
P1	P2	D1	P4	D2	D3	D4	P8	D5	D6	D7	D8

Parity bits are placed at positions 1, 2, 4 & 8.

The error position is calculated using parity-check results,

Error position = P1 + 2P2 + 4P4 + 8P8

If the result is 0, there is no error.

otherwise, it has an error.

```
        else:
            print("No error detected")

        data = [
            h[3], h[5], h[6], h[7],
            h[9], h[10], h[11], h[12]
        ]
        binary = ''.join(map(str, data))
        return chr(int(binary, 2))

text = input("Enter text: ")
channel = ""
for ch in text:
    binary = format(ord(ch), '08b')
    channel += hamming_encode(binary)

with open("channel.txt", "w") as file:
    file.write(channel)
print("\nEncoded data:")
print(channel)

with open("channel.txt", "r") as file:
    received = file.read()
print("\n Data received from channel:")
print(received)
pos = int(input("\n Enter bit position to introduce
        error (0 for no error): "))
```

```
if pos != 0:
    bits = list(received)
    bits[pos - 1] = '1' if bits[pos - 1] == '0' else '0'
    received = ''.join(bits)
    print("Error introduced at position:", pos)

output = ""
for i in range(0, len(received), 12):
    block = received[i : i+12]
    output += hamming_decode(block)
print("\n original message:")
print(output)
```

Sample I/P:

Enter text: Hello

Enter bit position to introduce error
(0 for no error): 10

Sample O/P:

Encoded data:

.....

Data saved in channel.txt

Data received from channel:

.....

Error introduced at position: 10

No error detected

Error detected at position: 10

Error corrected

No error detected

No error detected

No error detected

original message:

Hello

Result:

Thus, the Hamming code error detection and correction program was successfully implemented in python. The input text was converted into binary and encoded with redundant parity bits. A single-bit transmission error was successfully detected, corrected and the original msg was recovered.

8/8/2006 7. Implement flow control at data link layer using sliding window Protocol

Aim:
To implement sliding window Protocol using python for flow control at data link layer and simulate frame transmission using ACK and NACK.

Algorithm:
(i) Read the window size from user.
(ii) Read a texts message
(iii) Divide the message into frames, with one character in each frame.
(iv) Send frames according to window size.
(v) Receiver checks the frame number.
(vi) If the frame is correct, send ACK
(or) else send NACK.
(vii) on receiving ACK, slide the window forward and on receiving NACK, retransmit the required frame and do the above steps for remaining frames.

Python code:

```
import time
window = int(input("Enter window size: "))
message = input("Enter message: ")
frames = list(enumerate(message))
print("\nFrames:")
for no, data in frames:
    print(f"Frame {no}: {data}")
received = ""
i = 0
while i < len(frames):
    current = frames[i: i + window]
    print("\nSending:")
    for no, data in current:
        print(f"Frame {no}: {data}")
    time.sleep(1)
    for no, data in current:
        print(f"Receiver received frame {no}")
        error = input(
            f"Is frame {no} correct? (y/n): "
        )
        if error.lower() == 'y':
            print(f"ACK {no+1}")
            received += data
```

```
        else:
            print(f"NACK {rno}")
            print(f"Resending frame {rno}")
            time.sleep(1)
            print(f"Frame {rno} accepted")
            print(f"ACK {rno+1}")
            received += data

    i += window
print("All frames transmitted")
print("Received message:", received)
```

Sample I/P:

Enter window size: 3

Enter message: HELLO

Sample O/P:

Frames:

Frame 0: H

Frame 1 : E

Frame 2 : L

Frame 3 : L

Frame 4 : O

Sending:

Frame 0: H

Frame 1: E

Frame 2: L

Receiver received from Frame 0

Is frame 0 correct? (y/n): y

ACK 1

Receiver received from frame 1

Is frame 1 correct? (y/n): y

ACK 2

Receiver received from frame 2

Is frame 2 correct? (y/n): n

NACK 2

Resending Frame 2

Frame 2 accepted

Ack 3

Result:

Thus, the Sliding Window Protocol was successfully implemented in python. Frames were transmitted according to specified window size, and ACK/NACK was used for flow control.

9. IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER SIMULATOR

Aim:
To implement classless IP subnetting using Cisco Packet Tracer and configure routers, switches, and PCs with appropriate IP addresses and subnet masks to establish network connectivity.

Procedure:

(i) open Cisco Packet Tracer and creates a new workspace.

(ii) Place the required routers, switches, and PCs and then connect it using ethernet cable.

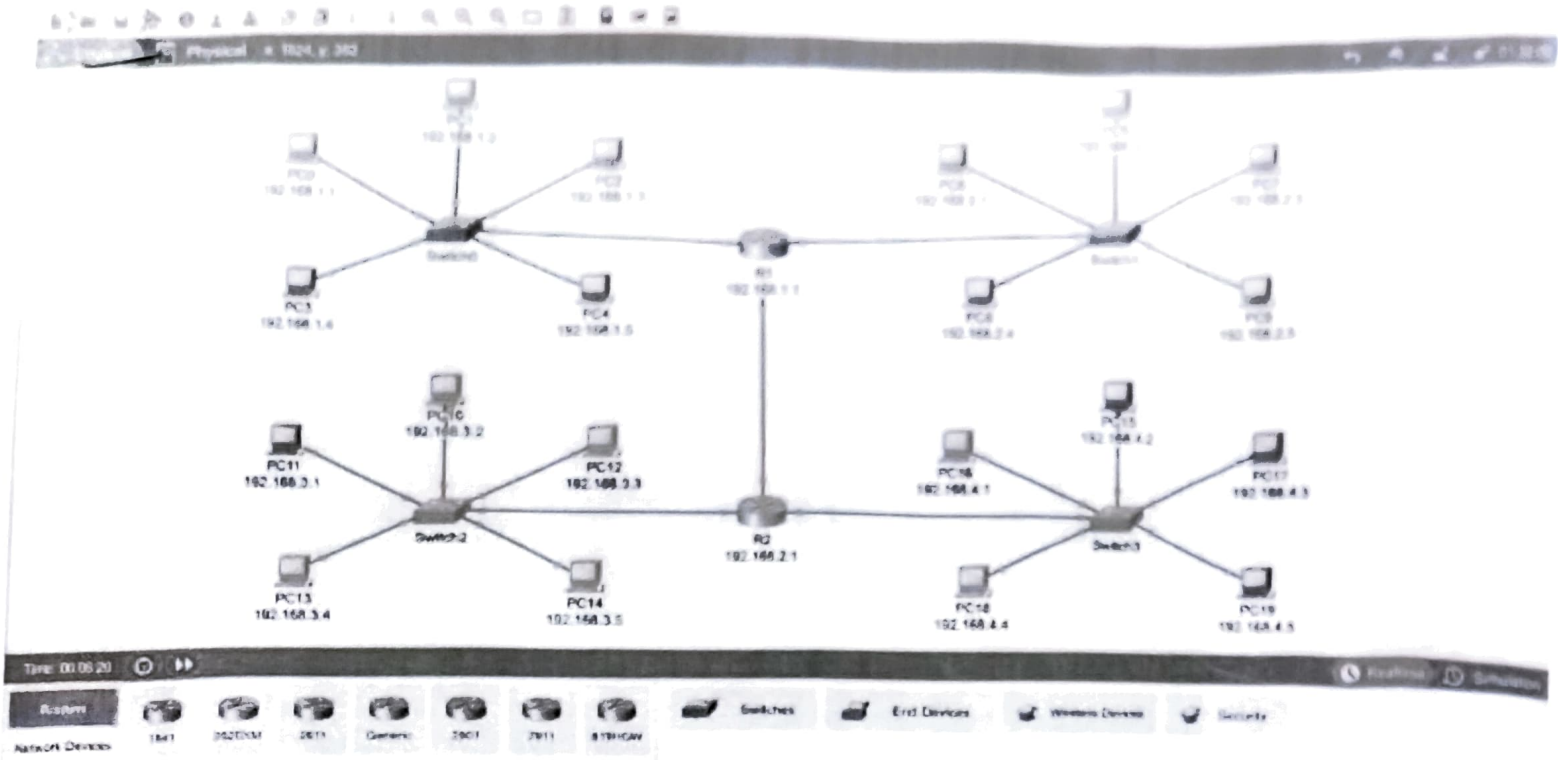
(iii) use subnet mask 255.255.255.224 (/27) for subnetting

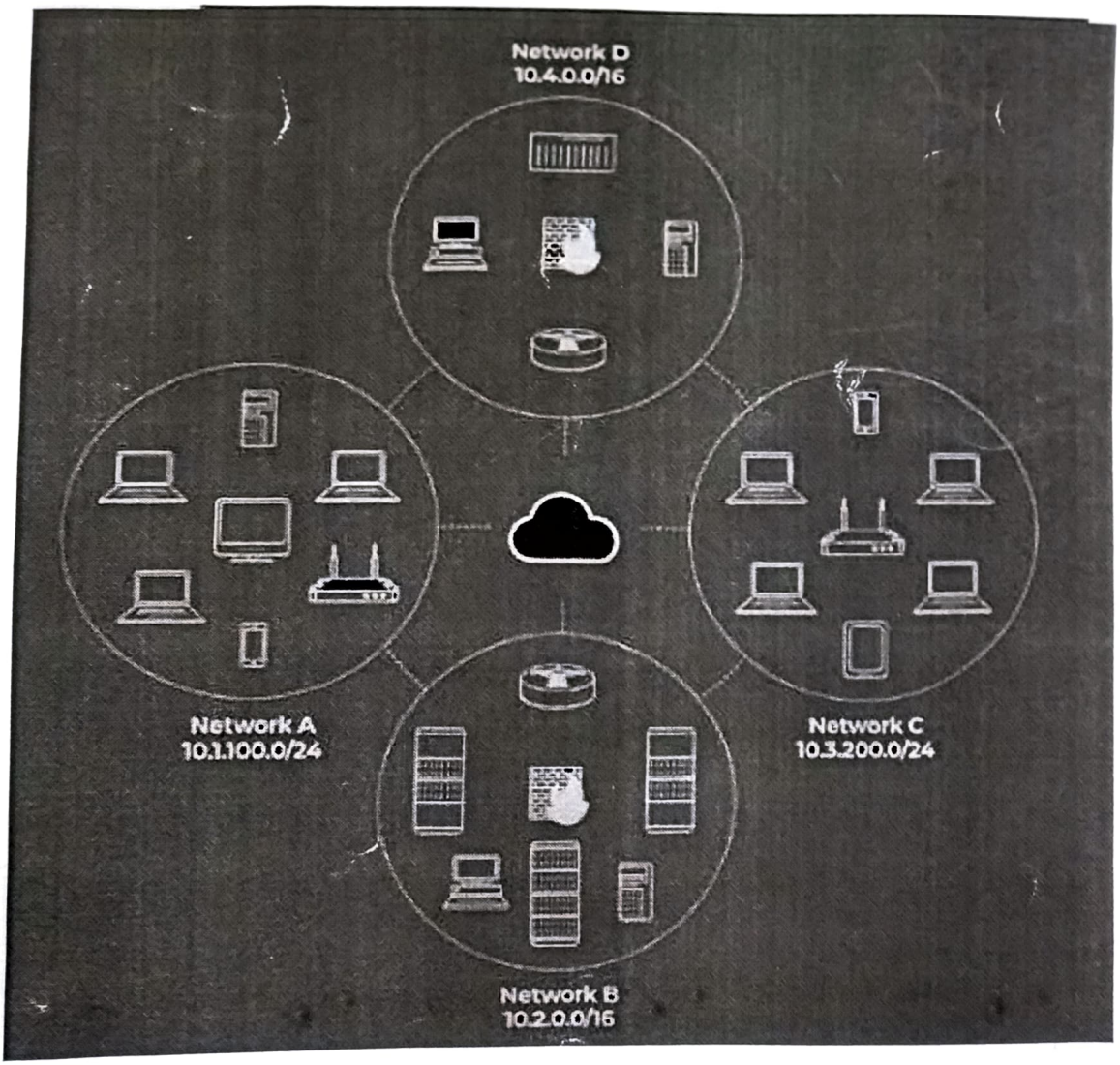
(iv) configure the route interfaces through CLI:
- ★ Enter privileged mode using enable
- ★ Enter configuration mode using configure terminal
- ★ Select required interface
- ★ Assign the IP address as subnet mask.
- ★ Enable interface using no shutdown

(v) configure switch ports as access ports.

(vi) open the configuration window of each PC and assign,
- ★ IP address
- ★ Subnet mask
- ★ Default gateway

(vii) Repeat the configuration for all required subnets and check router information status, show ip interface brief


PC11
192.168.3.1
PC12
192.168.3.3
PC13
192.168.3.4
PC14
192.168.3.5
PC15
192.168.4.2
PC16
192.168.4.1
PC17
192.168.4.3
PC18
192.168.4.4
PC19
192.168.4.5
R2
192.168.2.1


Network D
10.4.0.0/16
Network A
10.1.100.0/24
Network C
10.3.200.0/24
Network B
10.2.0.0/16

(viii) open the PC command prompt and use the ping command to test connectivity between devices.

(ix) If the ping is successful, the subnetted network is functioning correctly.

Result:

The classless IP subnetting was successfully implemented using cisco Packet tracer. The 192.168.1.0/24 network was divided into smaller /27 subnets, and the routers, switches, and PCs were configured with appropriate IP addresses.

8a). Discover Live Hosts using Nmap

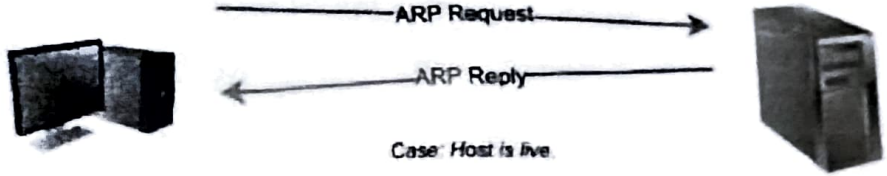
Aim:

To discover live hosts in a network using Nmap by performing ARP, ICMP, TCP and UDP-based host discovery on TryHackMe Nmap room.

Procedure:

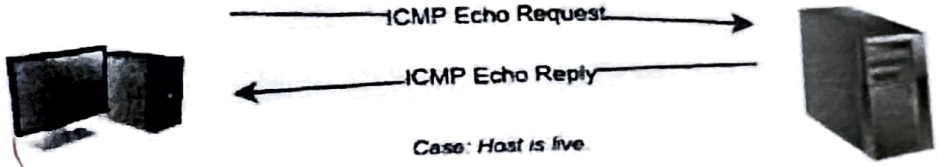
1. open the TryHackMe Nmap room and start the Attack Box.
2. Identify the target IP address/subnet to be scanned.
3. use Nmap host-discovery options to identify active systems.
4. Perform ARP-based discovery using nmap -PR -sn Target
5. Perform ICMP Echo discovery using -PE option and study ICMP Timestamp (-PP) and address mask (-PM) discovery.
6. Perform TCP SYN and TCP ACK host discovery using -PS and -PA and study UDP-based host discovery and observe ICMP port unreachable responses.
7. use reverse-DNS lookup options to obtain hostnames where applicable.
8. Record live hosts and verify the responses obtained from the scans.

nmap -PR -sn TARGET



Case: Host is live.

nmap -PE -sn TARGET



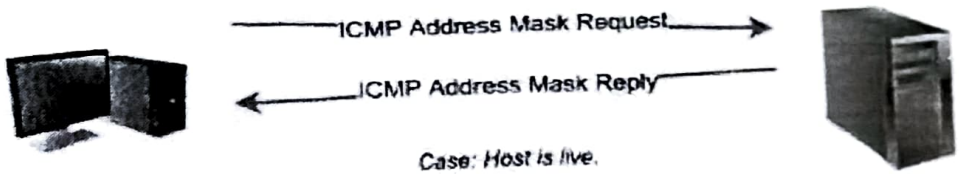
Case: Host is live.

nmap -PP -sn TARGET



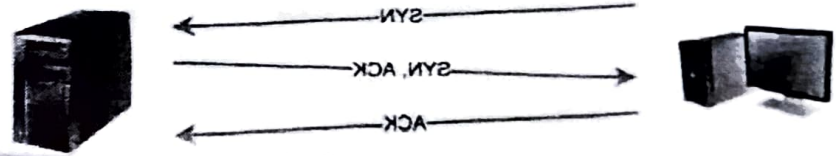
Case: Host is live.

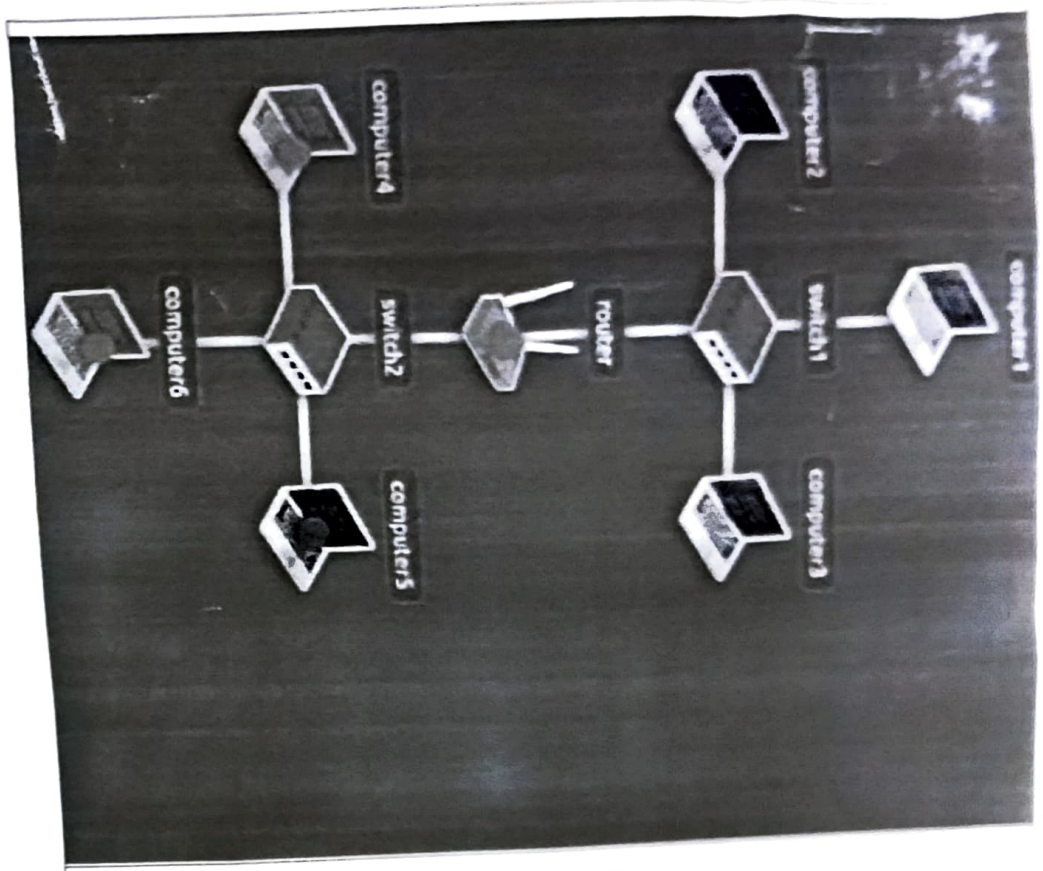
nmap -PM -sn TARGET



Case: Host is live.

TCP 3-Way Handshake





	ISO/OSI	TCP/IP	
7	Application Layer	Application Layer	HTTP, HTTPS, SMTP, POP3, IMAP, SSH, FTP, SNMP, Telnet, RDP
6	Presentation Layer		
5	Session Layer		
4	Transport Layer	Transport Layer	TCP, UDP
3	Network Layer	Network Layer	IPv4, IPv6, ICMP, IPsec
2	Data Link Layer	Link Layer	ARP, Ethernet, 802.3, 802.11, PPP, DSL, Bluetooth
1	Physical Layer		

Result:

Live hosts were successfully discovered using Nmap through ARP, ICMP, TCP and UDP host-discovery techniques. The practical demonstrated how different protocols can be used to determine whether a host is active.

8b). configuration of wireless LAN using Cisco Packet Tracer

Aim:

To design and configure a Wireless LAN using a Linksys wireless router and three PCs in Cisco Packet Tracer with static IP addressing and WEP security.

Procedure:

1. Place one Linksys Wireless Router and three PCs (PC0, PC1 and PC2) in Packet tracer.
2. configure the router SSID as MotherNetwork
3. Set the router IP address to 192.168.0.1 with subnet mask 255.255.255.0
4. configure WEP security and set the key as 0123456789.
5. configure static IP addresses:
   - ★ PC0 - 192.168.0.2
   - ★ PC1 - 192.168.0.3
   - ★ PC2 - 192.168.0.4
6. Set the subnet mask to 255.255.255.0 and default gateway to 192.168.0.1 for all PCs.
7. open PC wireless, refresh the available networks and select MotherNetwork.
8. Enter WEP key 0123456789 and connect each PC to the wireless router and verify wireless connection using ping.

Result:

The wireless LAN was successfully configured in cisco Packet Tracer. The three PCs were connected to the MotherNetwork using static IP addresses and secured with configured WEP key.

10). a Interconnecting with Routers

Aim:

To design and configure a single internetwork using a router in cisco Packet Tracer and verify connectivity between two PCs

Procedure:

1). open cisco Packet Tracer and place one router and two PCs in the workspace.

2). connect PC0 to FastEthernet 0/0 of router1 using a copper straight-through cable.

3). Apply the same for PC1 to FastEthernet 0/1 and open the router1 CLI and enter privileged mode using enable command.

4). Configure FastEthernet 0/0 with IP address 192.168.10.1 and subnet mask 255.255.255.0 and enable FastEthernet 0/0 using no shutdown command.

5). configure FastEthernet 0/1 with IP address 192.168.20.1 and subnet mask 255.255.255.0 and enable it.

6). configure PC0 with IP 192.168.10.2 and gateway 192.168.10.1 and configure PC1 with IP 192.168.20.2 and gateway 192.168.20.1.

Result:

Thus, a simple internetwork was successfully designed and configured using a router in cisco Packet Tracer, and connectivity between PC0 and PC1 was

Router0

Physical Config

IOS Command Line Interface

```
Press RETURN to get started!


Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z
Router(config)#interface FastEthernet0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to
up
interface FastEthernet0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to
up

Router#
%SYS-5-CONFIG_I: Configured from console by console
```

Copy Paste

PC0

Physical Config Desktop Custom Interface

IP Configuration

X

IP Configuration

○ DHCP ◉ Static

IP Address	192.168.10.2
Subnet Mask	255.255.255.0
Default Gateway	192.168.10.1
DNS Server	

IPv6 Configuration

○ DHCP ○ Auto Config ◉ Static

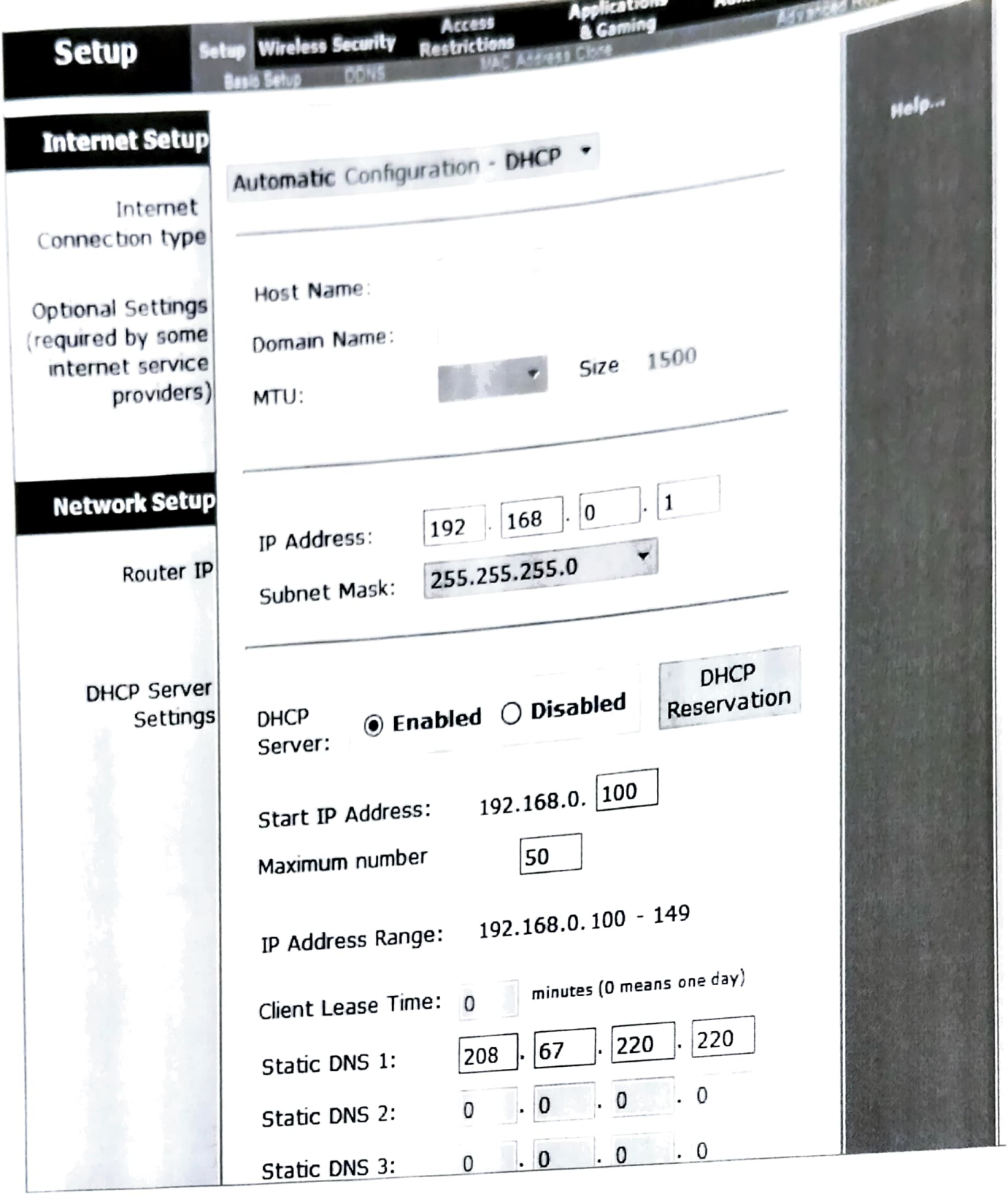
IPv6 Address	/
Link Local Address	FE80::201:42FF:FE97:C1DE
IPv6 Gateway	
IPv6 DNS Server	

10b). Wireless Router, DHCP server and Internet cloud

Aim:
To design and configure an internetwork using a wireless router, DHCP server and internet cloud in cisco Packet Tracer.

Procedure:
1). open cisco packet tracer and place a PC, laptop, wireless router, cable modem, internet cloud and cisco.com server.
2). connect PC to wireless router using copper-straight through cable.
3). connect wireless router to cable modem and cable modem to the internet cloud.
4). connect internet cloud to cisco.com server using required cables.
5). configure the wireless router with SSID HomeNetwork and enable its DHCP server.
6). configure laptop with wireless WPC300N module and connect it to HomeNetwork.
7). use ipconfig/release and ipconfig/renew on PC to obtain network configuration from DHCP.
8). test connectivity by using the command ping cisco.com and verify received replies.

Result:
Thus, the internetwork using a wireless router, DHCP server and internet cloud was successfully designed and configured, and connectivity to server was successfully verified.

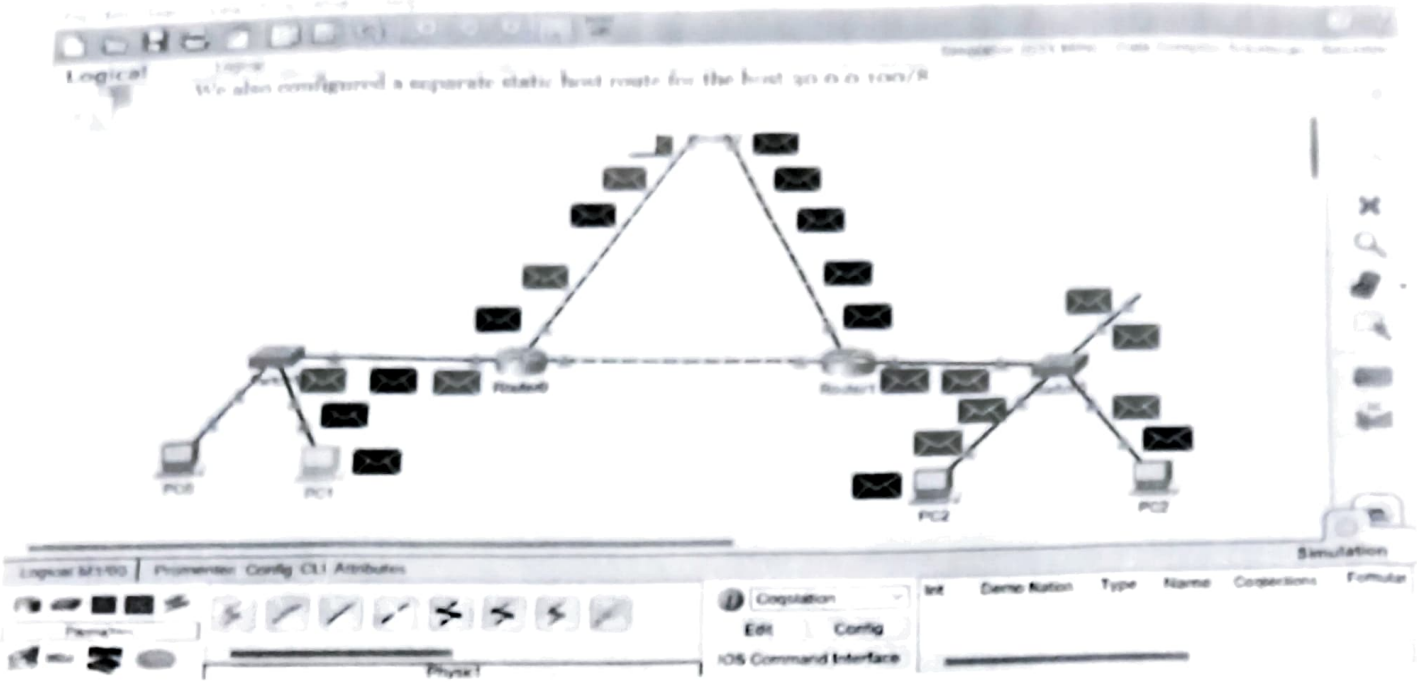

Setup
Setup
Wireless Security
Access Restrictions
Applications & Gaming
Basic Setup
DDNS
MAC Address Clone
Help...
Internet Setup
Internet Connection type
Automatic Configuration - DHCP
Optional Settings (required by some internet service providers)
Host Name:
Domain Name:
MTU:
Size 1500
Network Setup
Router IP
IP Address: 192 . 168 . 0 . 1
Subnet Mask: 255.255.255.0
DHCP Server Settings
DHCP Server: Enabled Disabled
DHCP Reservation
Start IP Address: 192.168.0. 100
Maximum number 50
IP Address Range: 192.168.0.100 - 149
Client Lease Time: 0 minutes (0 means one day)
Static DNS 1: 208 . 67 . 220 . 220
Static DNS 2: 0 . 0 . 0 . 0
Static DNS 3: 0 . 0 . 0 . 0

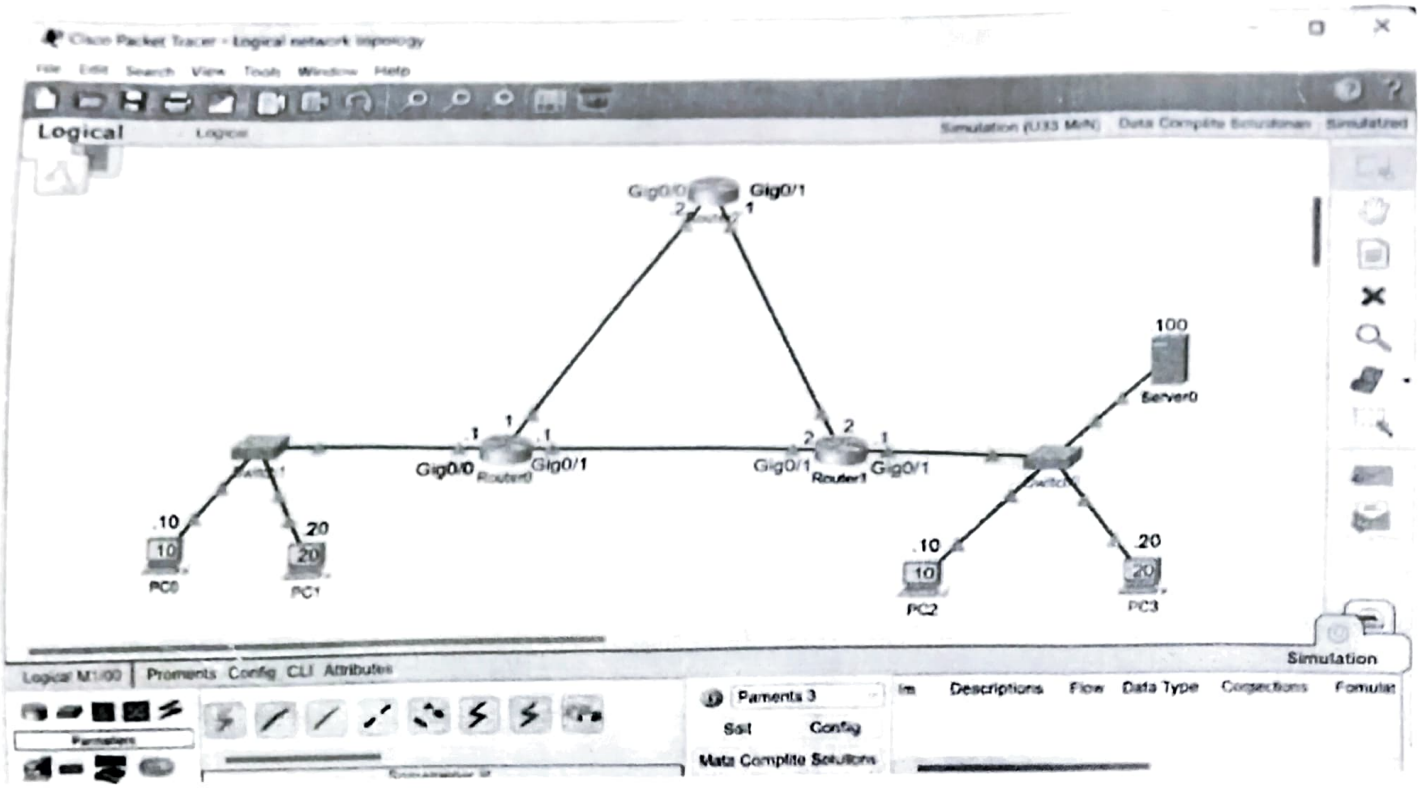
11 a). Simulate Static Routing configuration using Cisco Packet Tracer

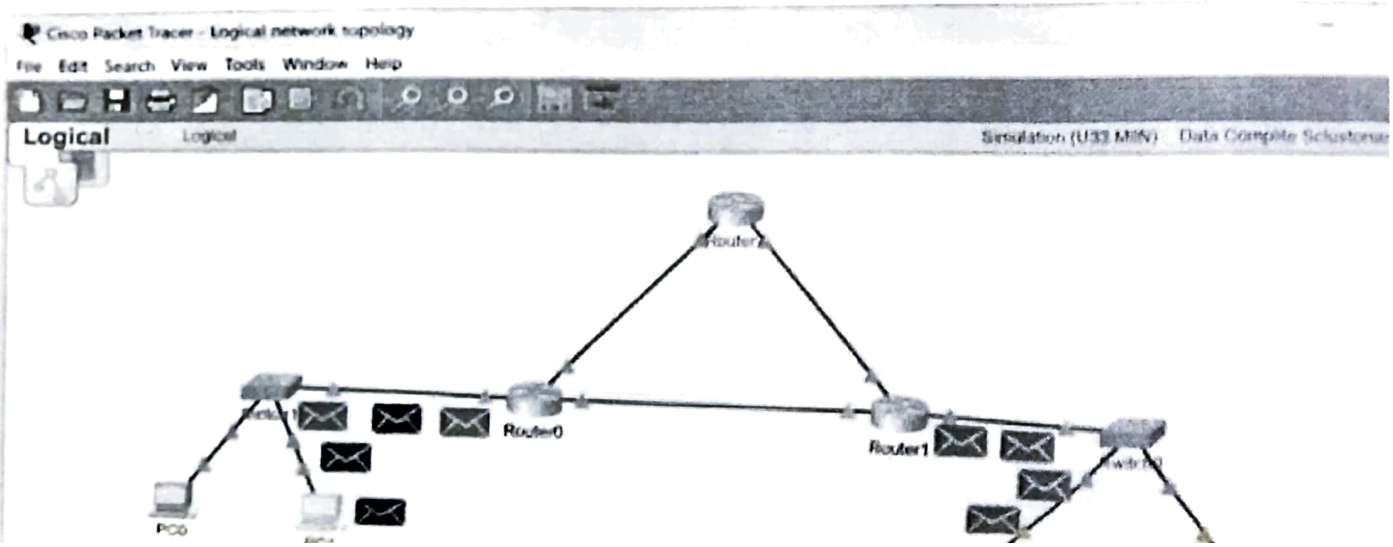
Aim:
To simulate Static Routing configuration using Cisco Packet Tracer and verify the connectivity between different networks using primary and backup routes.

Procedure:

1. Open Cisco Packet Tracer and create required network topology with PCs and routers.
2. Assign the given IP addresses and subnet masks to all PC and router interfaces.
3. Enable the router interfaces using no shutdown command.
4. Configure static routes to remote networks using ip route command.
5. Configure primary and backup routes using different Administrative Distance (AD) values.
6. Use show ip route static to verify the configured static routes & use ping and tracert commands to test connectivity.
7. Disconnect the primary link between the routers to simulate a link failure.
8. Check the routing table again and observe that the backup route becomes active.
9. Verify the connectivity again using ping and tracert after the link failure.


Logical
Router0
Router1
PC0
PC1
PC2
PC2
Simulation


Cisco Packet Tracer - Logical network topology
File Edit Search View Tools Window Help
Logical
Gig0/0
Gig0/1
Router0
Router1
Server0
100
PC0
PC1
PC2
PC3
10
20
Simulation


Cisco Packet Tracer - Logical network topology
File Edit Search View Tools Window Help
Logical
Router0
Router1
PC0

Result:

Thus, static routing was successfully simulated using Cisco Packet Tracer. Primary and backup routes were configured using Administrative distance, and the backup route was successfully used when the primary route failed.

11b). Routing Information Protocol

Aim:

To configure RIP routing using cisco packet tracer

Procedure:

1. open cisco packet tracer and create the required router and PC topology.
2. connect the devices using suitable cables and assign IP addresses to PCs & router interfaces.
3. Enable router interfaces using no shutdown and enter RIP configuration mode using router rip.
4. Add the connected networks using network commands.
5. configure RIP on the required routers and check the routing table.
6. Test the connectivity using 'Ping' command.

Result:

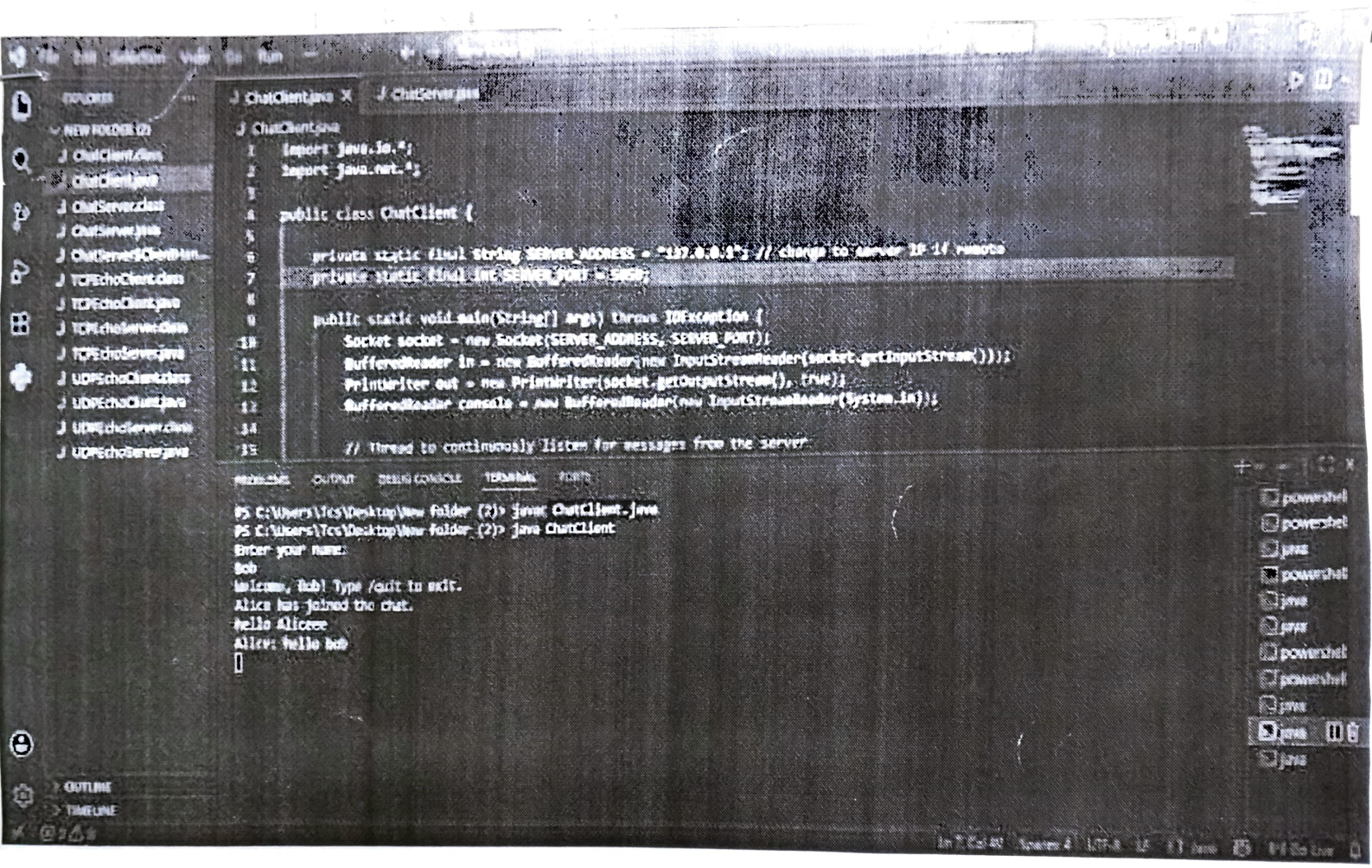
Thus, RIP routing was successfully configured and connectivity between the networks was verified.

12a). Echo client-server

Aim:
To implement an echo client-server using TCP/UDP sockets.

Procedure:
1). create the server socket and bind it to an IP address and port.
2). start listening for client requests and create the client socket.
3). connect the client to the server and enter a message from the client.
4). Send the message to the server.
5). Server receives and echoes the message and client receives the echoed message.
6). close the sockets.

Result:
Thus, the echo client-server was successfully implemented using sockets.


ChatClient.java
import java.io.*;
import java.net.*;
public class ChatClient {
private static final String SERVER_ADDRESS = "127.0.0.1"; // change to server IP if remote
private static final int SERVER_PORT = 5000;
public static void main(String[] args) throws IOException {
Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader console = new BufferedReader(new InputStreamReader(System.in));
// Thread to continuously listen for messages from the server
PS C:\Users\Tcs\Desktop\New folder (2)> javac ChatClient.java
PS C:\Users\Tcs\Desktop\New folder (2)> java ChatClient
Enter your name:
Bob
Welcome, Bob! Type /exit to exit.
Alice has joined the chat.
Hello Alicee
Alice: Hello Bob

12.b). chat client-server

Aim:

To implement a chat program using socket programming.

procedure:

1). create the server socket and bind it to an IP address and port.

2). start listening for a client and create the client socket.

3). Establish connection between client and server and enter a message from the client.

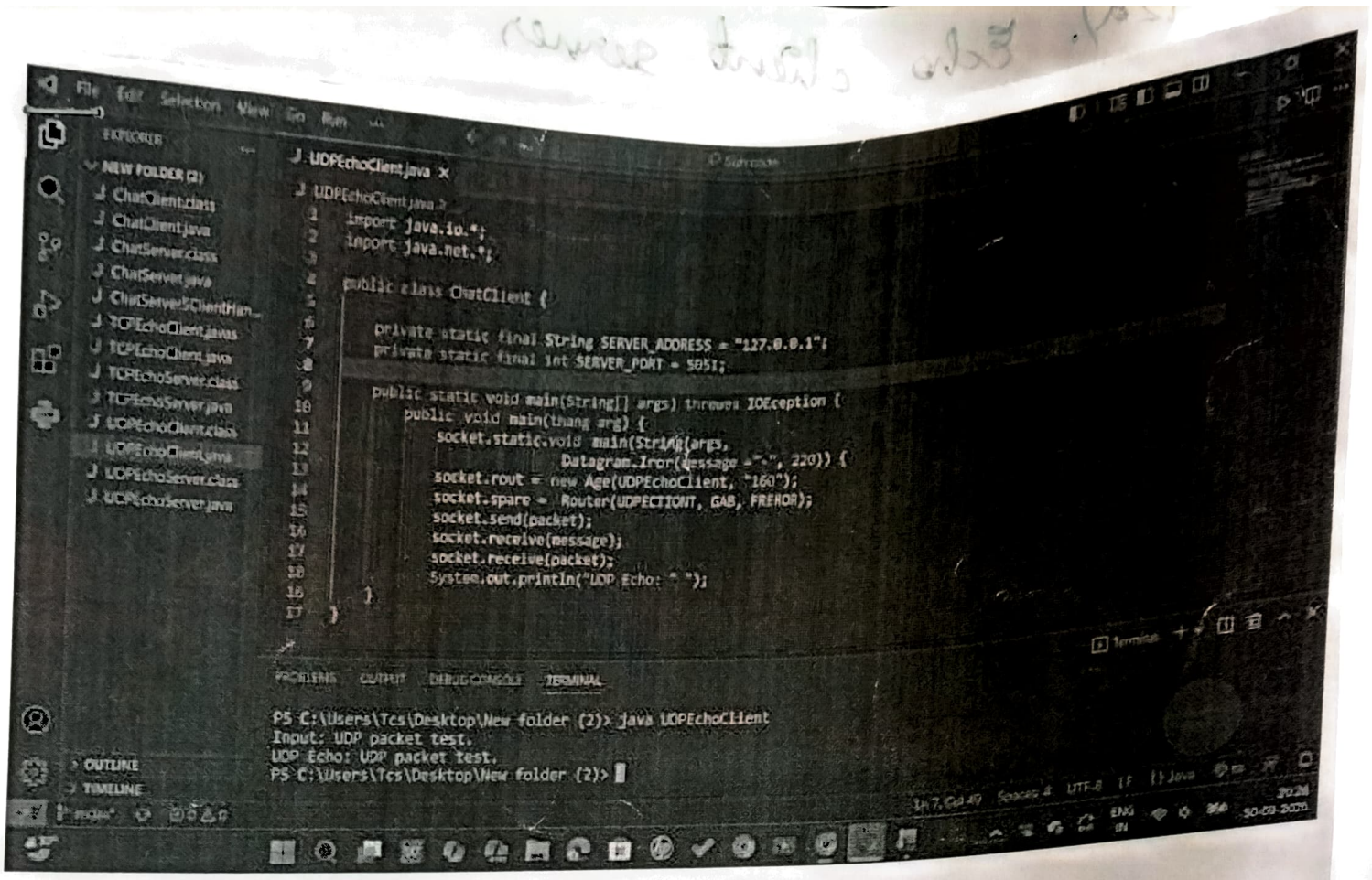
4). Send and receive messages between client and server.

5). Display the received messages and continue communication until termination.

6). close the sockets.

Result:

Thus, the chat client-server was successfully implemented using socket programming.


File Edit Selection View Go Run
EXPLORER
NEW FOLDER (2)
ChatClient.class
ChatClient.java
ChatServer.class
ChatServer.java
TCPEchoClient.java
TCPEchoServer.class
TCPEchoServer.java
UDPEchoClient.class
UDPEchoClient.java
UDPEchoServer.class
UDPEchoServer.java
UDPEchoClient.java
import java.io.*;
import java.net.*;
public class ChatClient {
private static final String SERVER_ADDRESS = "127.0.0.1";
private static final int SERVER_PORT = 5051;
public static void main(String[] args) throws IOException {
public void main(thang arg) {
socket.static.void main(String[]args,
Datagram.Iror(message = "", 220)) {
socket.rout = new Age(UDPEchoClient, "160");
socket.spare = Router(UDPECTIONT, GAB, FREHOR);
socket.send(packet);
socket.receive(message);
socket.receive(packet);
System.out.println("UDP Echo: ");
}
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS C:\Users\Tcs\Desktop\New folder (2)> java UDPEchoClient
Input: UDP packet test.
UDP Echo: UDP packet test.
PS C:\Users\Tcs\Desktop\New folder (2)>
OUTLINE
TIMELINE

13. Implement your own ping program

Aim:

To implement a simple Ping program using socket programming to check whether a destination host is reachable and to calculate the response time.

Algorithm:

1). Start the program and get the destination IP address/hostname from user.

2). create raw socket using ICMP protocol.

3). construct an ICMP echo request packet and record the time before sending the packet.

4). Send ICMP packet to the destination host and wait for ICMP echo reply.

5). Record the time when the reply is received.

6). calculate the Round Trip Time (RTT) and display whether the host is reachable.

7). Repeat the process for required no. of packets and close the socket and terminate the program.

Python program:

```
import socket
import struct
import time
import os

def checksum(data):
    total = 0
    for i in range(0, len(data)-1, 2):
        total += (data[i] << 8) + data[i+1]
    if len(data)%2:
        total += data[-1] << 8
    total = (total >> 16) + (total & 0xffff)
    total += total >> 16
    return (~total) & 0xffff

def ping(host):
    try:
        dest_addr = socket.gethostbyname(host)
        sock = socket.socket(socket.AF_INET,
                             socket.SOCK_RAW,
                             socket.IPPROTO_ICMP)
        packet_id = os.getpid() & 0xffff
        sequence = 1
        header = struct.pack("!BBHHH", 8, 0, 0,
                             packet_id, sequence)
        data = b"PING"
        chksum = checksum(header + data)
        header = struct.pack("!BBHHH", 8, 0, chksum,
                             packet_id, sequence)
        packet = header + data
        start = time.time()
        sock.sendto(packet, (dest_addr, 1))
        sock.settimeout(2)
        sock.recvfrom(1024)
        end = time.time()
```

Result:

Thus, a ping program was implemented successfully using socket programming to determine host reachability and calculate round-trip time.

14. Packet sniffing using Raw Sockets

Aim:

To implement a packet sniffing program using raw sockets and capture network packets to analyze their basic information such as source address, destination address and protocol.

Algorithm:

=> Start the program and create a raw socket for capturing network packets.

=> Bind the socket to the required network interface, if necessary.

=> Receive packets continuously from network and extract the IP header from each packet.

=> Identify the source IP address and destination IP address.

=> Determine the protocol type such as TCP, UDP (or) ICMP and display packet information.

=> continue capturing packets until the user stops the program.

=> close the raw socket and terminate the program.

Result:

Thus, a packet sniffing program was implemented successfully using raw sockets, and the captured packets were analyzed based on their source IP, destination IP and protocol.

Python program:

```
import socket
import struct
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                     socket.IPPROTO_IP)
print("Packet sniffer started")
print("Waiting for packets")
while True:
    packet, address = sock.recvfrom(65535)
    ip_header = packet[0:20]
    iph = struct.unpack("!BBHHHBBH4s4s",
                        ip_header)

    source_ip = socket.inet_ntoa(iph[8])
    destination_ip = socket.inet_ntoa(iph[9])
    protocol = iph[6]

print("Source IP:", source_ip)
print("Destination IP:", destination_ip)
print("Protocol:", protocol)
print("Packet Size:", len(packet), "bytes")
```

O/P: Packet sniffer started

Packet 1
Source IP: 192.168.1.10
Destination IP: 142.250.183.14
Protocol: TCP

Packet 2
Source IP: 192.168.1.10
Destination IP: 8.8.8.8
Protocol: UDP

15. Analysis of various types of servlets using webalizer

Aim:

To analyze the web server log files using the Webalizer tool and study various server statistics such as number of requests, visitors, transferred data, URLs and response status codes.

Algorithm:

⇒ Start the webalizer application and install and configure webalizer on system.

⇒ Locate the web server access log file and configure webalizer with required log-file path and output directory.

⇒ Run webalizer on the selected log file.

⇒ Webalizer reads and analyzes the server log entries.

⇒ Generates statistics such as:

* No. of visits
* No. of requests
* Pages accessed
* Data transferred
* HTTP response codes
* Referring sites

⇒ open the generated HTML report in web browser

⇒ Analyze the server activity and repeat the analysis for different log files.

I/P:

access.log

O/P:

Webalizer Statistics:

Total requests	: 1250
Total visits	: 480
Total pages	: 920
Total bytes	: 15.4 MB
Successful requests	: 1180
404 errors	: 45



7/8

Result:

Thus, the Webalizer tool was used successfully to analyze web server log files, and various server statistics such as requests, visitors, pages, bandwidth usage and HTTP response codes were obtained.